

UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

Dipartimento di Scienze Fisiche, Informatiche e Matematiche

Corso di Laurea Magistrale in Informatica (LM-18)

Implementazione di una pipeline per la generazione di mesh 3D a partire da modelli CAD 2D di planimetrie architettoniche

Relatore:
Prof. Nicola Capodieci
Correlatore:
Prof. Fabio Pellacini

Candidato:
Simone Vaiasinni
Matricola 201711

ANNO ACCADEMICO 2025/2026

Abstract

I modelli CAD 2D costituiscono la rappresentazione più diffusa delle planimetrie architettoniche, ma la loro trasformazione in 3D, richiede tipicamente un intervento manuale. Questo lavoro di tesi presenta l'implementazione di una pipeline per la generazione automatica di mesh 3D a partire da planimetrie CAD.

La principale difficoltà nell'affrontare questo problema risiede nell'assenza di uno standard univoco tra i modelli CAD prodotti da autori e software diversi, il che rende necessario introdurre alcune assunzioni progettuali sulla struttura del modello di input, riguardanti in particolare la geometria dei muri e la denominazione di layer e blocchi.

A partire da un file DXF ben formato, la pipeline esegue il parsing delle entità geometriche, la ricostruzione dei varchi di porte e finestre, l'estrazione di un grafo planare tramite struttura dati Half-Edge e la classificazione delle facce risultanti, la triangolazione e l'estrusione verticale differenziata per tipo di elemento, per poi esportare la mesh in formato GLTF, completa di normali e coordinate texture. Il modello viene successivamente importato in Blender, dove viene arricchito con materiali, asset di porte e finestre, posizionati automaticamente sulla base delle informazioni sui varchi esportate dalla pipeline, ed elementi di scena, ai fini della produzione di rendering fotorealistici tramite il motore Cycles.

La pipeline è stata validata su tre planimetrie reali di complessità variabile, reperite da fonti pubbliche, ottenendo in tutti i casi una ricostruzione geometrica corretta a fronte di un intervento manuale limitato e circoscritto alla sola fase di preprocessing, con tempi di elaborazione della pipeline dell'ordine di pochi secondi.

Indice

Abstract **iii**

1 Introduzione **1**

1.1 Il dominio del problema 1

1.2 I problemi dei modelli CAD 1

1.3 Obiettivi della tesi 2

1.4 Struttura della tesi 3

2 Background teorico **4**

2.1 Strutture dati spaziali 4

2.1.1 Approcci alla ricerca di prossimità 4

2.1.2 L'hash spaziale 5

2.1.3 Complessità e scelta della dimensione della cella 5

2.2 Clustering di punti 6

2.2.1 L'algoritmo DBSCAN 7

2.2.2 Scelta dei parametri 10

2.3 Rappresentazione di una mesh 10

2.3.1 Topologia della mesh 11

2.3.2 Indexed Mesh 11

2.3.3 Grafi planari e Planar Straight-Line Graph 11

2.3.4 Strutture dati per la connettività 12

2.3.5 Attributi dei vertici 14

2.4 Algoritmi di triangolazione 15

2.4.1 Ear clipping 16

2.4.2 Triangolazione di Delaunay 16

3 Progettazione e implementazione della pipeline **19**

3.1 Assunzioni progettuali 19

3.2 Preprocessing 19

3.3 Parsing 23

3.4 Collasso dei vertici 27
3.5 Ricostruzione dei varchi 30
3.5.1 Ricostruzione porte 30
3.5.2 Ricostruzione finestre 31
3.6 Estrazione e classificazione delle facce 34
3.7 Ricostruzione mesh 36
3.7.1 Pavimento e soffitto 37
3.7.2 Porte 37
3.7.3 Finestre 37
3.8 Esportazione 37

4 Rendering **40**
4.1 Generazione della scena 40
4.2 Setup della scena e rendering 41
4.3 Risultati 41

5 Casi di studio e risultati **45**
5.1 Secondo caso di studio 45
5.2 Terzo caso di studio 51

6 Conclusioni **55**
6.1 Sintesi del lavoro 55
6.2 Limiti 55
6.3 Sviluppi futuri 56

Riferimenti **57**

1. Introduzione

L'obiettivo del presente lavoro di tesi è quello di sviluppare un sistema in grado di generare automaticamente una mesh 3D partendo da un modello CAD 2D di una planimetria di una singola stanza o di un appartamento. Per ogni modello CAD 2D che verrà letto in input dal sistema il risultato sarà una mesh 3D con vertici, indici, normali e coordinate di texture che possono essere immediatamente utilizzate per il rendering.

1.1 Il dominio del problema

Il dominio del problema comprende i piani architettonici CAD 2D in formato DXF (Drawing Exchange Format). Mentre DWG e DXF sono standard de facto nel campo del CAD, sono diversi tra loro per quanto riguarda l'approccio alla codifica: DWG è un formato binario proprietario, più efficiente nello spazio ma meno leggibile in quanto può essere letto da poche applicazioni, mentre DXF è un formato basato sul testo (in ASCII) sviluppato in modo tale da non presentare problemi di interoperabilità con i software di terzi.

Per poter iniziare a lavorare sulla progettazione della pipeline di elaborazione è stato prima necessario raccogliere un dataset composto di modelli CAD 2D: sono stati scaricati diversi modelli dai seguenti siti `draftsperson.net`, `pincad.com` e `archweb.com`. Questi modelli erano inizialmente disponibili nel formato DWG e sono stati quindi convertiti in formato DXF e controllati con un software di editing come LibreCAD. Tra i modelli disponibili ne sono stati selezionati alcuni come studi di caso, utilizzati nel corso della tesi per illustrare le diverse fasi della pipeline.

1.2 I problemi dei modelli CAD

Il formato DXF non fornisce direttamente poligoni, bensì un insieme di entità geometriche disconnesse tra loro, quali `POLYLINE`, `LINE`, `ARC` ecc., organizzate in layer e blocchi. È qui che emerge la prima criticità: non esiste alcuno standard che imponga una convenzione univoca sui nomi dei layer o dei blocchi. Un architetto può decidere di nominare i layer in un certo modo, e un altro architetto in un modo diverso. Questo vale anche per i blocchi, che molto spesso presentano nomi privi di significato.

Un secondo problema riguarda la rappresentazione geometrica degli elementi. Per quanto riguarda i muri, ad esempio, un architetto può rappresentarle con un semplice contorno privo di spessore, mentre un altro le disegna con uno spessore definito. Alcuni utilizzano polilinee che definiscono contorni già chiusi, altri invece semplici segmenti che spesso si intersecano impropriamente tra loro e risultano aperti, facendo apparire la geometria come "rotta".

Non solo i muri, ma anche le finestre presentano una notevole variabilità: alcuni architetti le descrivono come blocchi complessi composti da centinaia di segmenti, altri come semplici segmenti che collegano due pareti, altri ancora come rettangoli definiti da quattro lati, con ulteriori segmenti interni per rappresentare il vetro.

Le porte, invece, tendono a essere rappresentate in modo più uniforme tra i vari modelli, tipicamente tramite archi; in alcuni modelli, tuttavia, in particolare per le porte d'ingresso, possono essere rappresentate tramite polilinee o segmenti anziché tramite un arco.

Questa eterogeneità è la ragione principale per cui non è possibile progettare un algoritmo generale in grado di elaborare correttamente qualsiasi modello CAD. È necessaria, a monte, una fase di preprocessing volta a rendere il modello quanto più consistente possibile (Sezione 3.2 del Capitolo 3).

1.3 Obiettivi della tesi

Il presente lavoro si pone i seguenti obiettivi:

- progettare un parser in grado di interpretare in modo robusto le entità geometriche eterogenee presenti in un file DXF (segmenti, polilinee, archi, blocchi), riconducendole a un insieme coerente di muri, porte e finestre;
- ricostruire la topologia della planimetria a partire da primitive geometriche, unificando i vertici duplicati e ricostruendo correttamente i varchi sui muri;
- generare una mesh 3D completa di normali e coordinate texture, distinguendo pavimento, soffitto, muri, porte e finestre;
- esportare il modello in formato GLTF pronto per essere importato in Blender, così da effettuare un rendering fotorealistico della scena.

1.4 Struttura della tesi

Il resto della tesi è organizzato come segue.

Il Capitolo 2 introduce le nozioni di geometria computazionale necessarie per comprendere le scelte progettuali descritte nei capitoli successivi: strutture dati spaziali, algoritmi di clustering di punti, grafi planari, tecniche di triangolazione di poligoni e rappresentazione di mesh 3D (in particolare normali e coordinate texture).

Il Capitolo 3 descrive la progettazione e l'implementazione della pipeline, dalle assunzioni progettuali fino a ciascuna delle fasi che la compongono: preprocessing, parsing, collasso dei vertici, ricostruzione dei varchi, estrazione e classificazione delle facce, triangolazione ed estrusione, esportazione.

Il Capitolo 4 conclude il lavoro descrivendo la fase di rendering: generazione della scena in Blender e setup dei materiali, delle luci, della camera e del motore Cycles.

2. Background teorico

2.1 Strutture dati spaziali

Molti algoritmi di elaborazione geometrica richiedono, come operazione di base, di rispondere in modo efficiente a interrogazioni di prossimità tra punti: dato un punto p, trovare tutti i punti p' che distano da esso meno di una soglia r fissata. Questo problema è noto in letteratura come *fixed-radius near-neighbor problem* e può essere formalizzato come segue:

$$\text{dato } p, \text{ trovare } \{p' : d(p, p') < r\}$$

dove d è la distanza euclidea. Un approccio diretto (*naive*) consiste nel confrontare ogni punto con tutti gli altri, con una complessità $O(N)$ per ogni singola interrogazione e quindi $O(N^2)$ per elaborare un intero insieme di N punti: un costo che diventa rapidamente insostenibile al crescere della dimensione dell'insieme. Per questo motivo, quando il numero di interrogazioni è elevato, è preferibile appoggiarsi a una struttura dati pensata per organizzare i punti nello spazio e restringere la ricerca ai soli candidati plausibili.

2.1.1 Approcci alla ricerca di prossimità

Le strutture dati più diffuse per questo tipo di problema si dividono in due grandi famiglie. La prima è quella delle strutture gerarchiche di partizionamento dello spazio, come il *K-D Tree* o il *quadtree*, che suddividono ricorsivamente lo spazio in regioni via via più piccole organizzate in un albero. Queste strutture garantiscono interrogazioni di vicinanza in tempo $O(\log N)$.

La seconda famiglia è quella delle strutture basate su una griglia regolare, tra cui l'*hash spaziale* (*spatial hash*). A differenza delle strutture ad albero, un hash spaziale non organizza i punti in una gerarchia, ma li distribuisce in *bucket* (celle) di una griglia uniforme, usando le coordinate del punto stesso come chiave di hashing. Questo approccio è più semplice da implementare, supporta inserimenti e rimozioni in tempo pressoché costante, ed è particolarmente indicato quando il raggio di ricerca r è noto a priori [1].

2.1.2 L'hash spaziale

Il funzionamento di un hash spaziale si basa su un'idea semplice: lo spazio 2D viene suddiviso in una griglia regolare, in cui ogni cella ha una dimensione fissa. A ciascun punto viene associata esattamente una cella, calcolata a partire dalle sue coordinate. L'insieme dei punti che ricadono nella stessa cella viene poi memorizzato in una struttura hash (ad esempio una `unordered_map` o una `unordered_multimap`), la cui chiave è la coppia di indici che identifica la cella.

La proprietà che rende questa struttura efficiente per le interrogazioni di prossimità è la seguente: se la dimensione della cella è scelta pari al raggio di ricerca r, due punti che distano meno di r si trovano necessariamente nella stessa cella oppure in una delle celle immediatamente adiacenti. Questo significa che, per rispondere a un'interrogazione centrata in un punto p, non è necessario esaminare l'intero insieme di punti, ma è sufficiente calcolare la cella di appartenenza di p ed esaminare unicamente le celle vicine (una griglia 3×3), effettuando un confronto esplicito di distanza solo con i punti che vi ricadono.

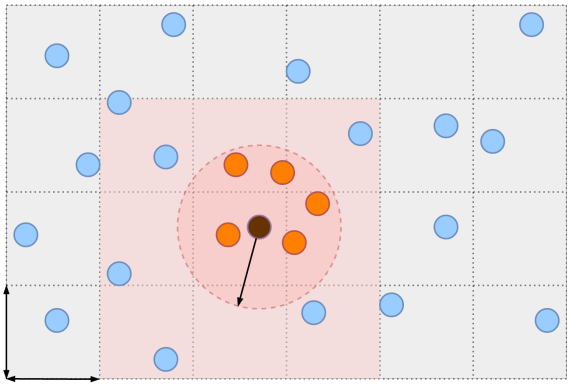


Figura 2.1: Rappresentazione di una griglia regolare per l'hash spaziale: i punti sono associati alle celle e la ricerca di prossimità esamina le celle adiacenti.

2.1.3 Complessità e scelta della dimensione della cella

L'inserimento di un nuovo punto ha complessità $O(1)$. Analogamente, un'interrogazione di prossimità ha complessità $O(k)$, dove k è il numero medio di punti contenuti nelle celle esaminate, indipendente dalla dimensione totale N. Nel caso peggiore, in cui i punti si concentrano in modo fortemente non uniforme (ad esempio tutti nella stessa cella), la complessità può degradare fino a $O(N)$; tale scenario è raro nella pratica.

La dimensione della cella è un parametro cruciale per le prestazioni della struttura. Se le celle sono più grandi del raggio r, il numero di celle da esaminare resta uguale ma cresce il numero di punti presenti in ciascuna cella, aumentando i confronti da effettuare. Se, al contrario, le celle sono più piccole di r, il numero di punti per cella si riduce, ma bisogna estendere la ricerca a un intorno più ampio (ad esempio una griglia 5×5). La scelta di dimensionare la cella pari a r rappresenta, in generale, il miglior compromesso tra il numero di celle da visitare e il numero di punti da confrontare all'interno di ciascuna.

Questa stessa tecnica è nota in letteratura anche con il nome di *Cell Lists*, e le sue proprietà teoriche sono state analizzate in dettaglio da Tollander de Balsch [2], che ne formalizza la complessità considerando non la singola interrogazione ma l'elaborazione dell'intero insieme di N punti. Nell'ipotesi che il numero di punti per cella sia limitato da una costante indipendente da N (i punti non si accumulano in una stessa posizione), il numero totale di confronti necessari a individuare tutte le coppie di punti vicine cresce linearmente, $O(N)$, contro $O(N^2)$ dell'approccio brute force; nel caso peggiore, in cui tutti i punti ricadono in un'unica cella, le due complessità coincidono.

Questa struttura dati verrà ripresa e applicata concretamente nella Sezione **??**, nell'ambito dell'operazione di collasso dei vertici (*vertex snapping*) durante l'elaborazione della planimetria CAD.

2.2 Clustering di punti

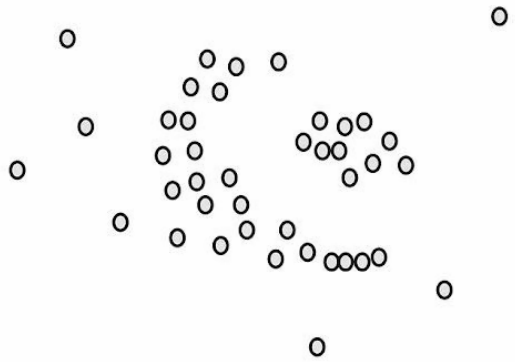
Un altro problema abbastanza comune nell'elaborazione di dati geometrici è quello di raggruppare un insieme di punti in sottoinsiemi (*cluster*) coerenti, in base alla loro posizione. A differenza della ricerca di prossimità, che risponde a interrogazioni puntuali, il clustering spaziale ha l'obiettivo di partizionare l'intero insieme di dati, individuando automaticamente quali punti appartengono allo stesso oggetto e quali, invece, sono isolati o rappresentano rumore.

La categoria degli algoritmi più adatta è quella degli *density-based*, in particolare **DBSCAN** (*Density-Based Spatial Clustering of Applications with Noise*), introdotto da Ester et al. [5]. A differenza di algoritmi come *k-means*, che richiedono di specificare a priori il numero di cluster e tendono a produrre cluster di forma convessa, DBSCAN definisce i cluster come regioni di spazio ad alta densità di punti separate da regioni a

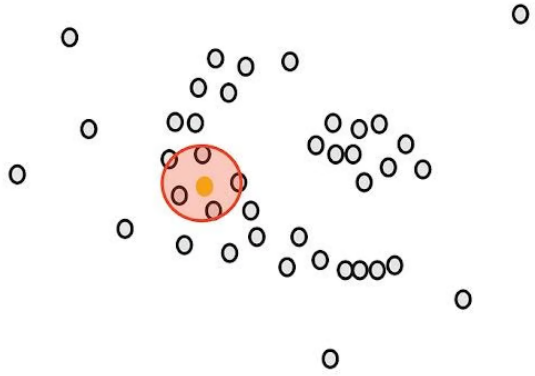
bassa densità, ed è quindi in grado di identificare cluster di forma arbitraria e di isolare automaticamente i punti isolati come rumore [13].

2.2.1 L'algoritmo DBSCAN

Prendo come riferimento l'articolo di Okan Yenigün [13]. Per capire come funziona l'algoritmo DBSCAN consideriamo il seguente esempio:

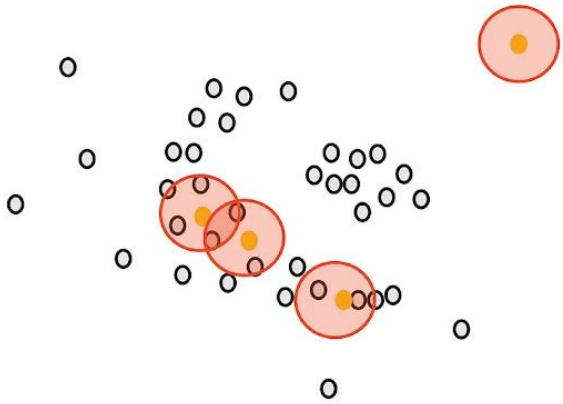


L'obbiettivo è di raggruppare questi punti in gruppi (cluster). Prendiamo inizialmente un punto da cui partire e disegnamo un cerchio attorno:

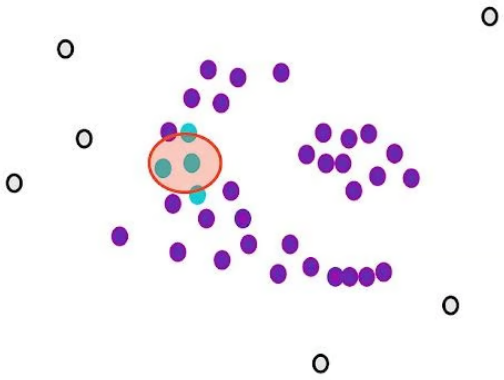


Il raggio ϵ del cerchio è il primo parametro da determinare. Dopodiché contiamo quanti punti si trovano all'interno del cerchio e vediamo che ci sono esattamente 5 punti di prossimità.

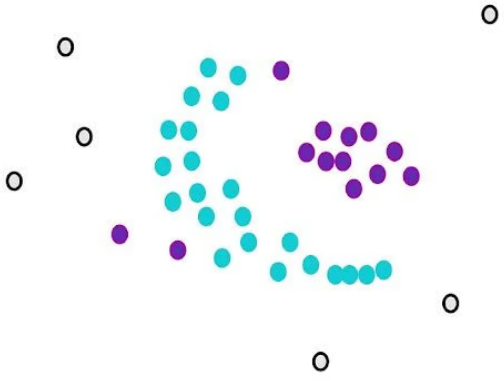
Adesso dobbiamo determinare un altro parametro, ovvero il numero minimo m di punti. Un punto si dice essere un "core point" se è vicino almento a m punti. Se consiriamo $m = 3$, allora i seguenti punti evidenziati in viola sono core point, mentre il punto in giallo no.



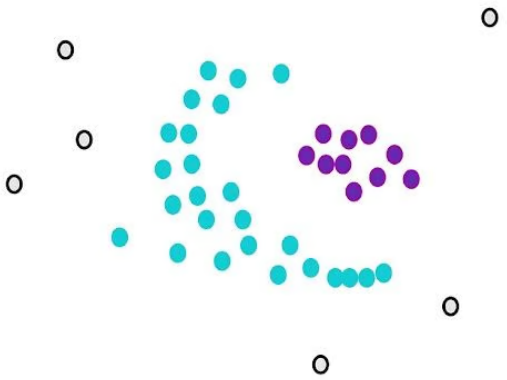
Dopo, selezioniamo a caso un core point e lo assegniamo come primo punto nel nostro primo cluster. I punti che si trovano vicino al core point vengono considerati dentro al cluster.



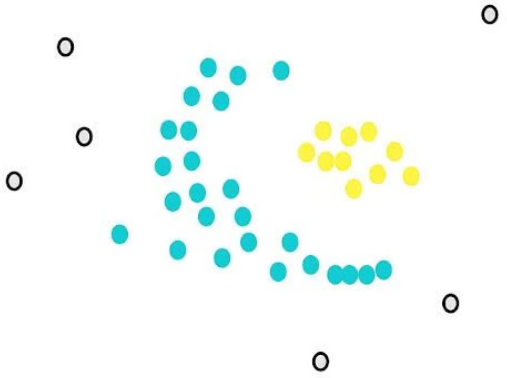
Ripetiamo il procedimento anche per gli altri punti vicini e ci fermiamo quando non possiamo più assegnare più assegnare core points al primo cluster.



Ci sono dei punti che sono vicini al primo cluster che però non sono dei core points. Deisegnamo i cerchi anche su questi punti e vediamo se rientrano nel cluster. Questi punti sono i punti di frontiera (o anomalie).



Sono rimasti altri core points che non sono ancora stati assegnati. Ne selezioniamo uno a caso e ripetiamo il procedimento.



DBSCAN è un algoritmo sequenziale, è quindi importante notare che i punti non core verranno assegnati al primo cluster che soddisfa il requisito di vicinanza.

2.2.2 Scelta dei parametri

La qualità del risultato prodotto da DBSCAN dipende dalla scelta dei due parametri ϵ e m. Un valore di ϵ troppo grande tende a far collassare cluster distinti in un unico cluster; al contrario, un valore troppo piccolo produce una frammentazione eccessiva. Analogamente, m regola la densità minima richiesta per considerare una regione un cluster: valori più alti producono cluster più densi e robusti al rumore, ma rischiano di scartare cluster legittimi composti da pochi punti.

2.3 Rappresentazione di una mesh

La maggior parte dei modelli 3D utilizzati in applicazioni reali è composta da complessi di triangoli con vertici condivisi, comunemente detti *mesh triangolari*; gestire questa struttura in modo efficiente è determinante per le prestazioni di un programma grafico, sia in termini di occupazione di memoria su disco che a runtime [8]. Una mesh triangolare non è un insieme di triangoli tra loro indipendenti, ma una rete di triangoli connessi tramite vertici e archi condivisi a formare un'unica superficie continua: è proprio questa connettività condivisa, e non la semplice somma dei singoli triangoli, a permettere di rappresentare ed elaborare la mesh in modo più efficiente rispetto a una collezione di triangoli scorrelati.

L'informazione minima necessaria per definire una mesh triangolare è l'insieme dei triangoli, ciascuno espresso come tripla di vertici, e la posizione nello spazio di ciascun vertice. La quasi totalità delle applicazioni pratiche, tuttavia, richiede di associare ai vertici dati aggiuntivi necessari per operazioni come il texture mapping, lo shading o l'animazione. I dati associati ai vertici sono i più comuni: a ciascun vertice possono essere associati parametri di materiale, coordinate texture, o altro; tali valori vengono poi interpolati linearmente all'interno di ciascun triangolo, ottenendo così una funzione continua definita sull'intera superficie della mesh.

2.3.1 Topologia della mesh

L'idea che una mesh rappresenti una superficie, e non un insieme arbitrario di triangoli, può essere formalizzata attraverso vincoli sulla sua topologia, ovvero sul modo in cui i triangoli sono connessi tra loro, indipendentemente dalla posizione dei vertici. Il vincolo più semplice e più restrittivo è che la mesh sia una varietà (*manifold*): una mesh manifold non presenta buchi o giunzioni ambigue, e separa in modo netto lo spazio interno alla superficie da quello esterno [8]. Questa proprietà è alla base del funzionamento di molti algoritmi di elaborazione geometrica, tra cui la struttura dati Half-Edge discussa nella Sezione 2.3.4, che assume implicitamente che la mesh su cui opera sia manifold.

In molte applicazioni è inoltre necessario poter distinguere il lato "esterno" di una superficie da quello "interno", proprietà nota come orientazione. Per un singolo triangolo, l'orientazione è convenzionalmente definita in base all'ordine con cui i suoi vertici sono elencati: il lato anteriore del triangolo è quello da cui i tre vertici appaiono disposti in senso antiorario.

2.3.2 Indexed Mesh

I triangoli di una mesh possono essere memorizzati come entità indipendenti, ciascuna con i propri tre vertici: questo approccio, tuttavia, comporta la duplicazione dei vertici condivisi da più triangoli, con un evidente spreco di spazio. In alternativa, è possibile memorizzare una sola volta ciascun vertice distinto e far sì che ogni triangolo vi faccia riferimento tramite un indice: questa soluzione, nota come *indexed triangle mesh* (o *shared-vertex mesh*), evita la duplicazione dei dati di vertice ed è la forma di memorizzazione più diffusa nelle applicazioni pratiche.

2.3.3 Grafi planari e Planar Straight-Line Graph

Le strutture dati per la connettività discusse nel seguito derivano concettualmente dall'idea di rappresentare una mesh come un grafo: un insieme di vertici e di archi, a cui si aggiunge un insieme di facce definite come cicli ordinati di vertici o archi. Un grafo è detto planare se può essere disegnato su un piano senza che i suoi archi si intersechino al di fuori dei vertici condivisi; è detto planare embedded (o piano) quando tale disposizione è stata effettivamente fissata nello spazio. Quando, in aggiunta, gli archi sono segmenti

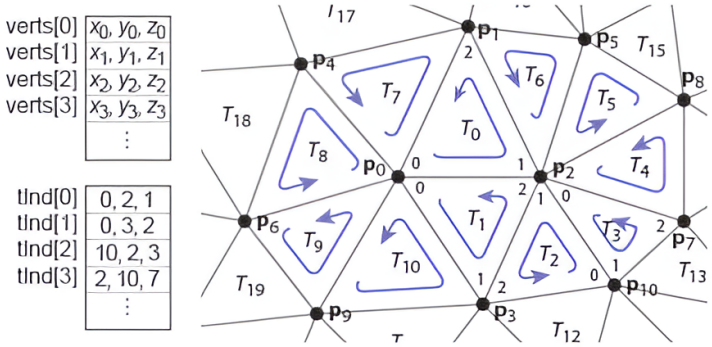


Figura 2.2: Rappresentazione di una *indexed triangle mesh*: i vertici sono memorizzati una sola volta e ogni triangolo vi fa riferimento tramite indici.

rettilinei anziché curve generiche, si parla di **Planar Straight-Line Graph** (PSLG): è la rappresentazione naturale per dati geometrici 2D costituiti da spezzate e poligoni, ed è tipicamente il punto di partenza per operazioni come l'estrazione delle facce racchiuse dal grafo o la sua triangolazione.

2.3.4 Strutture dati per la connettività

Oltre alla semplice memorizzazione indicizzata, esistono diverse strutture dati pensate per rendere esplicita ed efficiente la navigazione della connettività di una mesh, tanto più rilevanti quanto più le mesh coinvolte sono di grandi dimensioni [8].

Una prima soluzione, relativamente semplice, è la **Triangle-Neighbor Structure**: a partire da una mesh a vertici condivisi, si arricchisce ciascun triangolo con un riferimento ai tre triangoli adiacenti (uno per ciascun lato), e ciascun vertice con un riferimento a uno qualsiasi dei triangoli incidenti su di esso.

Una struttura più flessibile, ampiamente utilizzata, è la **Winged-Edge**, che memorizza l'informazione di connettività a livello di arco anziché di faccia [8]. In una mesh Winged-Edge, ogni arco memorizza un riferimento ai due vertici che collega (tipicamente indicati come vertice di testa e di coda), alle due facce di cui è parte (la faccia a sinistra e quella a destra), e, soprattutto, agli archi successivo e precedente nella percorrenza in senso antiorario di ciascuna delle due facce adiacenti. Ogni vertice e ogni faccia memorizza inoltre un riferimento a uno qualsiasi degli archi a esso incidenti. Il nome della struttura deriva dal fatto che l'insieme di questi riferimenti, disposti attorno a un singolo arco, assume graficamente la forma di una farfalla con quattro "ali".

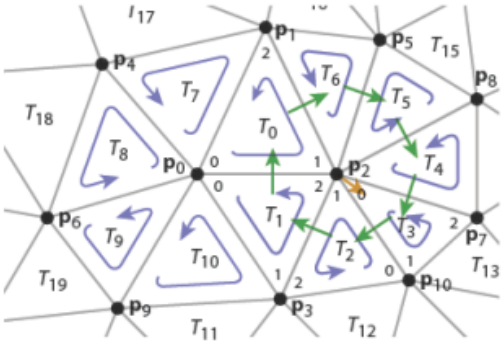


Figura 2.3: Rappresentazione della *Triangle-Neighbor Structure*: ogni triangolo referenzia i tre triangoli adiacenti (uno per lato) e ogni vertice referenzia uno dei triangoli incidenti.

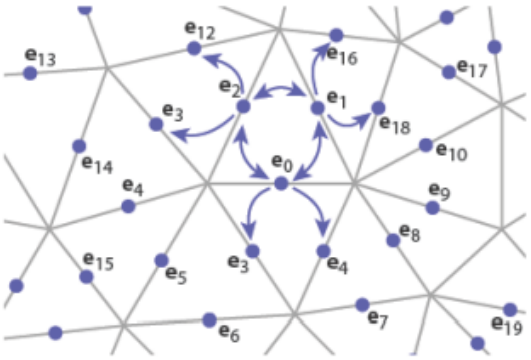


Figura 2.4: Rappresentazione della *Winged-Edge Structure*: ogni arco referenzia i due vertici, le due facce adiacenti e gli archi successivo e precedente nella percorrenza di ciascuna faccia, assumendo la caratteristica forma a farfalla.

La struttura Winged-Edge è elegante ma presenta un inconveniente pratico: per spostarsi da un arco al successivo è necessario verificare costantemente da quale lato lo si sta percorrendo, poiché un arco è condiviso da due facce e la sua orientazione dipende dal punto di vista. La struttura **Half-Edge** (o *doubly-linked edge list*) risolve questo problema suddividendo ciascun arco in due *half-edge* orientati in direzioni opposte, uno per ciascuna delle due facce che condividono l'arco: l'informazione normalmente memorizzata su un singolo arco Winged-Edge viene così distribuita tra i due half-edge, ciascuno coerente con l'orientazione della propria faccia. Ogni half-edge memorizza un riferimento alla faccia sul proprio lato dell'arco, al vertice verso cui punta, al proprio gemello (*twin*, l'half-edge opposto che rappresenta lo stesso arco fisico visto dall'altro lato) e all'half-edge successivo lungo il perimetro della propria faccia. Percorrendo ripetutamente questi riferimenti "successivi" si ottiene un ciclo chiuso corrispondente esattamente al contorno di una faccia, mentre alternando i riferimenti al gemello e al successivo è possibile iterare sugli archi che si dipartono da un dato vertice: proprietà che rende la struttura Half-Edge particolarmente indicata per algoritmi che devono navigare ripetutamente la topologia della mesh, come l'estrazione o la classificazione delle facce racchiuse da un PSLG.

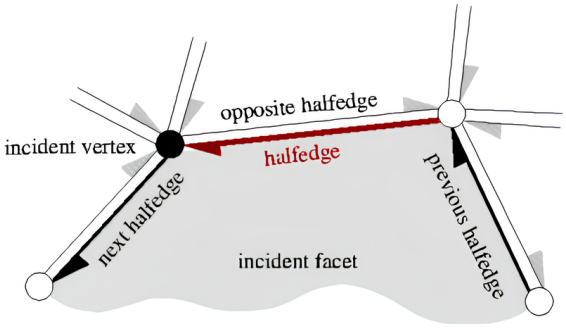


Figura 2.5: Rappresentazione della *Half-Edge Structure*: ogni arco è suddiviso in due half-edge opposti, ciascuno con riferimenti alla faccia, al vertice di destinazione, al gemello e all'half-edge successivo.

2.3.5 Attributi dei vertici

Come discusso in apertura di questa sezione, un vertice può essere arricchito con attributi opzionali oltre alla propria posizione nello spazio; tra questi, due sono di particolare rilevanza per una pipeline di rendering: il vettore normale e le coordinate texture.

Il vettore **normale** è un vettore unitario perpendicolare alla superficie nel punto considerato, ed è l'informazione che un motore di rendering utilizza per determinare l'orientazione della superficie rispetto a una sorgente luminosa ai fini del calcolo dello shading.

Le **coordinate texture** associano invece a ciascun vertice una posizione (u, v), con $u, v \in [0, 1]$, all'interno di un'immagine bidimensionale (la texture) usata per aggiungere dettaglio visivo alla superficie senza dover infittire la geometria della mesh. Per applicare correttamente una texture a un triangolo è necessario specificare, per ciascuno dei suoi tre vertici, quale punto della texture gli corrisponde; il resto della superficie del triangolo viene poi campionato per interpolazione tra questi tre valori. Per convenzione, la coordinata $(0, 0)$ corrisponde all'angolo inferiore sinistro dell'immagine e $(1, 1)$ all'angolo superiore destro. Qualora vengano specificate coordinate al di fuori dell'intervallo $[0, 1]$, il comportamento del campionamento dipende dalla modalità di *wrapping* scelta (ripetizione, ripetizione speculare, o clamping ai bordi), un dettaglio implementativo che esula dagli scopi di questa trattazione teorica.

2.4 Algoritmi di triangolazione

Un problema classico della computer grafica è quello di scomporre un poligono semplice in un insieme di triangoli i cui vertici sono solo quelli del poligono semplice. Per definizione, un poligono semplice è una sequenza ordinata di n punti. I vertici consecutivi sono collegati da un bordo e l'ultimo bordo collega il primo e l'ultimo punto. Ogni vertice condivide esattamente due spigoli. Gli unici punti in cui è consentito l'intersezione degli spigoli sono i vertici.

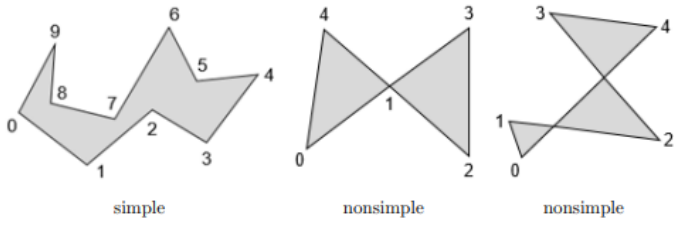


Figura 2.6: Rappresentazione di un poligono semplice: i vertici sono collegati in sequenza da spigoli che non si intersecano se non nei vertici stessi.

La scomposizione di un poligono semplice in triangoli è chiamata triangolazione del poligono. Qualsiasi triangolazione di un poligono semplice di n vertici ha $n-2$ triangoli.

Nel corso degli anni sono stati proposti numerosi algoritmi per risolvere il problema della triangolazione di un poligono, che si differenziano tra loro per generalità, complessità computazionale e qualità geometrica dei triangoli prodotti. In questa sezione se ne richiamano due tra i più rappresentativi: l'*ear clipping*, il più semplice, e la (*Constrained*) *Delaunay Triangulation*, in grado di produrre triangoli di qualità geometrica superiore.

2.4.1 Ear clipping

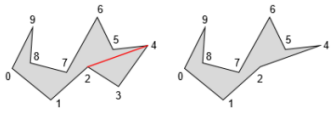
L'ear clipping è uno dei metodi più diretti per triangolare un poligono semplice. Il metodo si basa sulla nozione di *orecchio* (*ear*): dati tre vertici consecutivi del poligono, essi formano un orecchio se il vertice centrale è convesso e il segmento che unisce il primo e il terzo vertice è interamente contenuto all'interno del poligono, cioè se il triangolo da essi individuato può essere staccato dal poligono senza uscire dal suo contorno. Un risultato noto in geometria computazionale, il *teorema dei due orecchi*, garantisce che ogni poligono semplice con almeno quattro vertici possieda sempre almeno due orecchi.

Si individua un orecchio, lo si rimuove dal poligono (aggiungendo il triangolo corrispondente all'insieme di output), ottenendo un nuovo poligono, più piccolo, che soddisfa ancora le condizioni del teorema; l'operazione viene ripetuta fino a quando non rimane un solo triangolo.

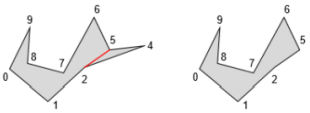
L'algoritmo è semplice da implementare ma esegue in tempo $O(n^2)$ [4]. Esistono varianti più efficienti, come la decomposizione in poligoni monotoni ($O(n \log n)$) o algoritmi randomizzati praticamente lineari, ma sono più complesse da implementare.

2.4.2 Triangolazione di Delaunay

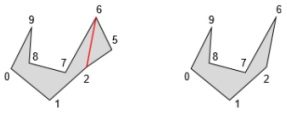
Un approccio alternativo, che si applica in generale a un insieme di punti anziché a un poligono, è la triangolazione di Delaunay (*Delaunay Triangulation*, DT): dato un insieme di punti nel piano, la DT suddivide il loro inviluppo convesso in triangoli tali che la circonferenza circoscritta a ciascuno di essi (la circonferenza passante per i suoi tre vertici) non contenga, al proprio interno, nessun altro punto dell'insieme. Questa proprietà ha l'effetto di massimizzare il più piccolo angolo tra tutti i triangoli della triangolazione, evitando così la formazione di triangoli eccessivamente sottili o degeneri (*sliver*).



(a) Poligono di partenza.



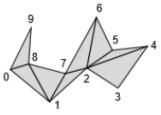
(b) Individuazione di un orecchio.



(c) Rimozione dell'orecchio e aggiunta del triangolo.



(d) Il poligono risultante è più piccolo.



(e) Triangolazione completata.

Figura 2.7: Illustrazione del procedimento di *ear clipping* su un poligono semplice: si individua iterativamente un orecchio, lo si rimuove e si aggiunge il triangolo corrispondente, fino a ottenere la triangolazione completa.

La DT dipende unicamente dalla posizione dei punti e non tiene conto di eventuali archi che si desidera vedere presenti nella triangolazione risultante: nulla impedisce che un triangolo la "attraversi". La Constrained Delaunay Triangulation (CDT) generalizza la DT introducendo la possibilità di imporre un insieme di segmenti obbligatori (*constraint*), che devono necessariamente comparire come archi della triangolazione finale. L'input al problema della CDT è tipicamente un PSLG: un insieme di punti e di segmenti non incidenti tra loro, questi ultimi trattati come vincoli da rispettare.

Per illustrare questa idea, la figura seguente mostra un esempio di ciò che accade quando un insieme di punti è collegato in modo arbitrario rispetto a uno conforme al criterio di Delaunay [7]:

Dal punto di vista della complessità computazionale, sono noti diversi algoritmi in grado di calcolare la CDT di un PSLG generico in tempo $O(n \log n)$; nel caso particolare in cui l'input sia un singolo poligono semplice, la CDT può essere calcolata in tempo lineare $O(n)$.

Rispetto all'ear clipping, la CDT presenta due caratteristiche importanti: da un lato tende a produrre triangoli di qualità geometrica superiore evitando la formazione di triangoli degeneri; dall'altro consente di specificare esplicitamente quali segmenti debbano

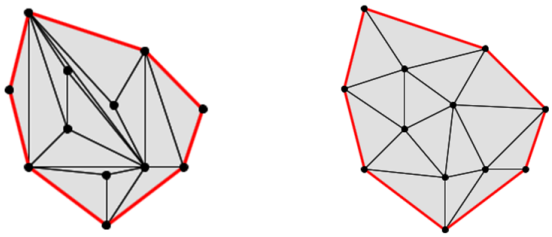


Figura 2.8: Confronto tra una connessione arbitraria dei punti e una triangolazione conforme al criterio di Delaunay. Nel secondo caso non ci sono triangoli degeneri.

necessariamente far parte della triangolazione risultante.

3. Progettazione e implementazione della pipeline

3.1 Assunzioni progettuali

La difficoltà principale di questo progetto risiede nell'assenza di uno standard univoco tra i modelli CAD, il che rende estremamente complesso progettare un algoritmo in grado di generalizzare correttamente a qualunque modello di input. Per questo motivo è stato necessario introdurre alcune assunzioni progettuali, che definiscono i vincoli entro cui la pipeline è garantita funzionare correttamente:

- I muri devono essere rappresentati come poligoni chiusi dotati di un proprio spessore. Non sono ammesse discontinuità nei contorni (muri "rotti" o aperti): eventuali imperfezioni di questo tipo devono essere corrette manualmente, prima dell'esecuzione della pipeline, tramite un software di editing CAD come LibreCAD.
- Porte e finestre devono essere rappresentate come aperture (fori) nei muri.
- I nomi dei layer e dei blocchi presenti nel file DXF devono rispettare una convenzione fissa:
  - **Layer**: `A-WALL`, `A-DOOR`, `A-WINDOW`;
  - **Blocchi**: `BLOCK-DOOR-[0-9]`, `BLOCK-WINDOW-[0-9]`.
- Gli elementi di arredo presenti nel modello CAD vengono ignorati: la pipeline prende in considerazione esclusivamente muri, porte e finestre.

3.2 Preprocessing

La fase di preprocessing ha l'obiettivo di rendere il modello CAD quanto più consistente possibile, prima che venga elaborato dal resto della pipeline. Si interviene correggendo manualmente i nomi di layer e blocchi, e riparando eventuali geometrie incomplete o ridondanti.

Come caso di studio si considera il seguente modello CAD, aperto in LibreCAD per una prima analisi (Figura 3.1).

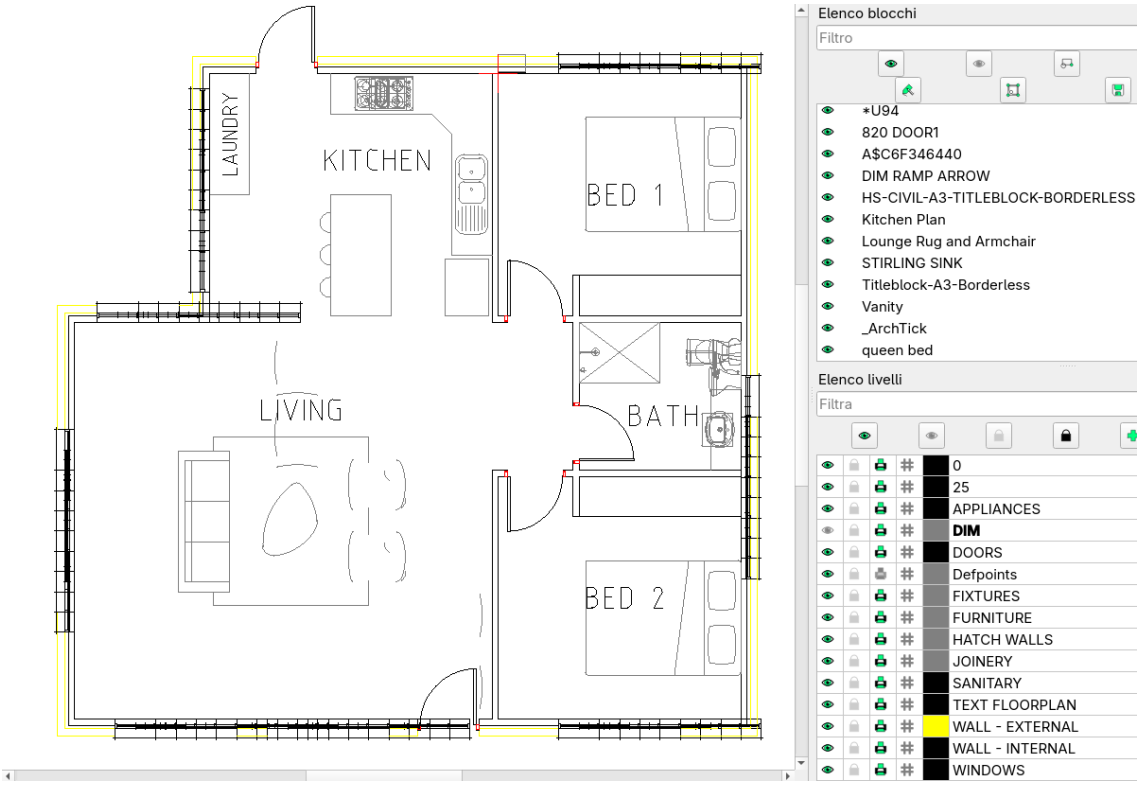


Figura 3.1: Modello CAD del caso di studio aperto in LibreCAD: sono visibili i layer e i blocchi con denominazioni non conformi ai criteri imposti.

Un primo problema riguarda la denominazione di layer e blocchi, che non rispetta la convenzione imposta nella Sezione 3.1; è quindi necessario rinominarli secondo i criteri richiesti.

Il modello presenta inoltre sia muri interni (layer `WALL-INTERNAL`) sia muri esterni (layer `WALL-EXTERNAL`). Questi ultimi risultano ridondanti e possono essere rimossi senza alcun impatto sulla geometria finale. Isolando i muri, si osserva come la loro geometria risulti "spezzata": i contorni formano poligoni aperti, in violazione delle condizioni di validità imposte nella Sezione 3.1, e richiedono pertanto una correzione manuale (Figura 3.2).

Anche le finestre presentano problemi di rappresentazione analoghi: ciascuna è composta da centinaia di segmenti, spesso sovrapposti tra loro; alcuni si estendono al di fuori della stanza, altri penetrano al suo interno.

Al termine della fase di correzione si ottiene il seguente risultato (Figura 3.3).

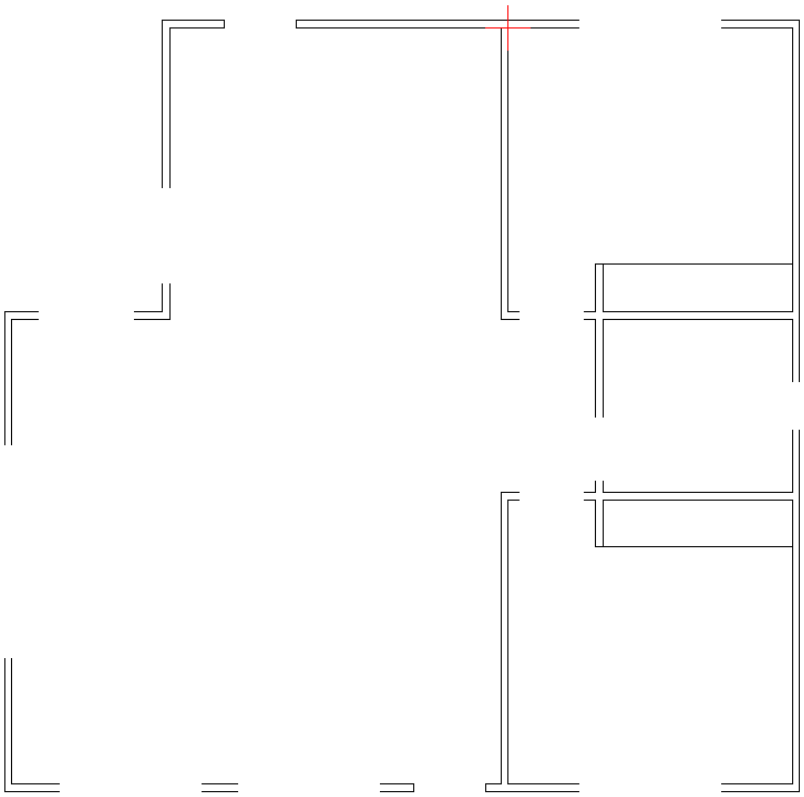


Figura 3.2: I muri isolati risultano geometricamente incompleti: formano poligoni aperti che non rispettano le condizioni di validità imposte e richiedono una correzione.

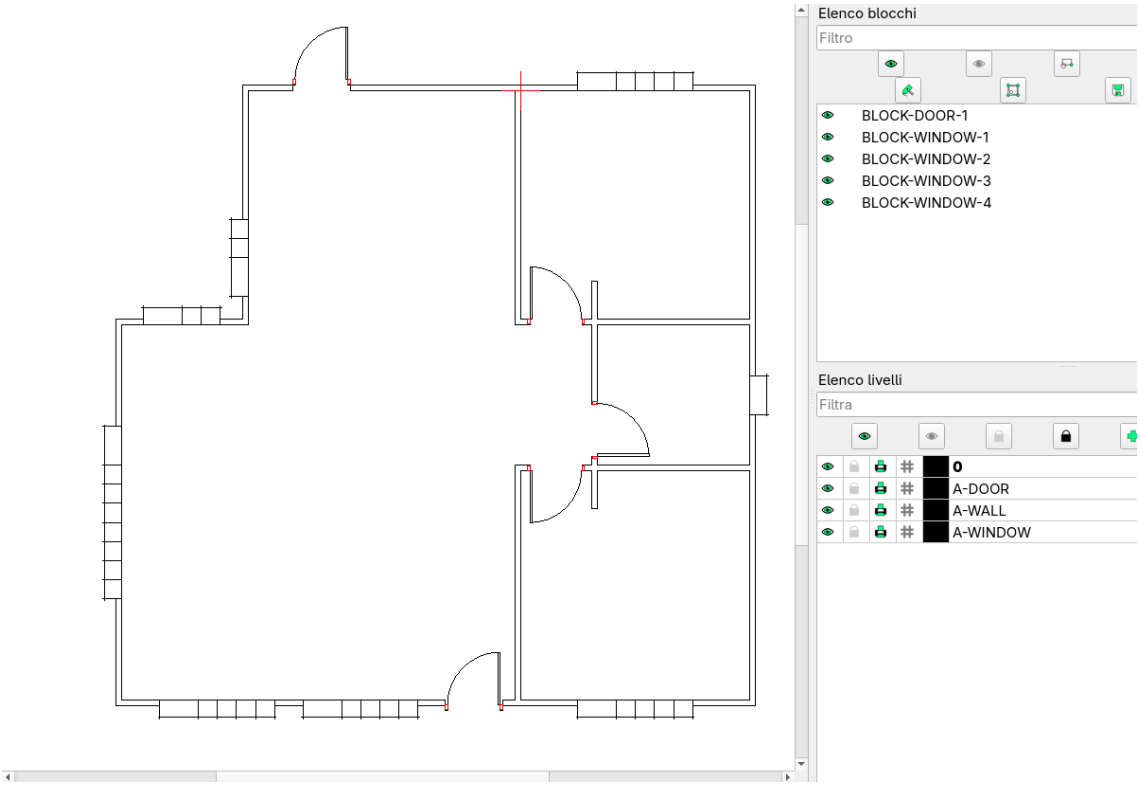


Figura 3.3: Modello CAD al termine della fase di preprocessing: i layer e i blocchi sono stati rinominati secondo i criteri imposti, i muri esterni ridondanti sono stati rimossi e la geometria dei muri e delle finestre è stata corretta.

3.3 Parsing

Il formato DXF organizza il disegno in primitive geometriche: `LINE`, `POLYLINE`, `ARC`, `CIRCLE`, ecc. Queste entità sono raggruppate in layer, che consentono di separare logicamente i diversi elementi del disegno (ad esempio muri, porte, finestre, arredi, testo). Oltre ai layer, il formato DXF supporta i blocchi (`BLOCK`), componenti riutilizzabili comunemente impiegati per rappresentare porte e finestre. Un blocco viene definito una volta all'interno del file e può essere istanziato più volte tramite entità `INSERT`, ciascuna con una propria posizione, scala e rotazione: questa funzionalità è particolarmente utile per rappresentare elementi ripetuti, come porte, finestre o arredi, che compaiono più volte nella stessa planimetria.

L'obiettivo della fase di parsing è estrarre da queste primitive le informazioni geometriche necessarie alla costruzione della mesh, collassando tutte le primitive sotto forma di segmenti, così da ottenere una rappresentazione uniforme.

Si consideri il modello CAD corretto durante il preprocessing (Sezione 3.2), che presenta layer standardizzati (`A-WALL`, `A-DOOR`, `A-WINDOW`) e blocchi rinominati (`BLOCK-DOOR-1`, `BLOCK-WINDOW-1`, `...`, `BLOCK-WINDOW-4`).

Il parsing viene realizzato in C++ mediante la libreria libdxfrw [3], che fornisce un'interfaccia per la lettura dei file DXF. Si definisce anzitutto la struttura di un segmento (Figura 3.4), rappresentato come coppia di vertici (posizioni nello spazio) e un tag che ne indica la semantica (muro, porta o finestra):

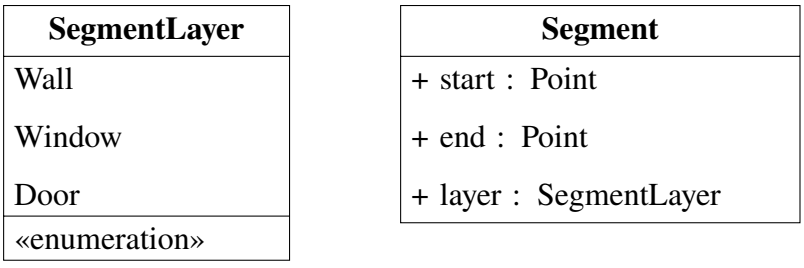


Figura 3.4: Diagramma UML delle entità `Segment` e `SegmentLayer`, che rappresentano rispettivamente un segmento geometrico e la sua semantica (muro, porta o finestra).

Questa classificazione per layer e per blocco è fondamentale perché, come si vedrà successivamente, consente di trattare in modo differenziato muri, porte e finestre durante

la fase di estrusione. La classe del parser (Figura 3.5) implementa le callback necessarie per gestire i diversi tipi di entità:

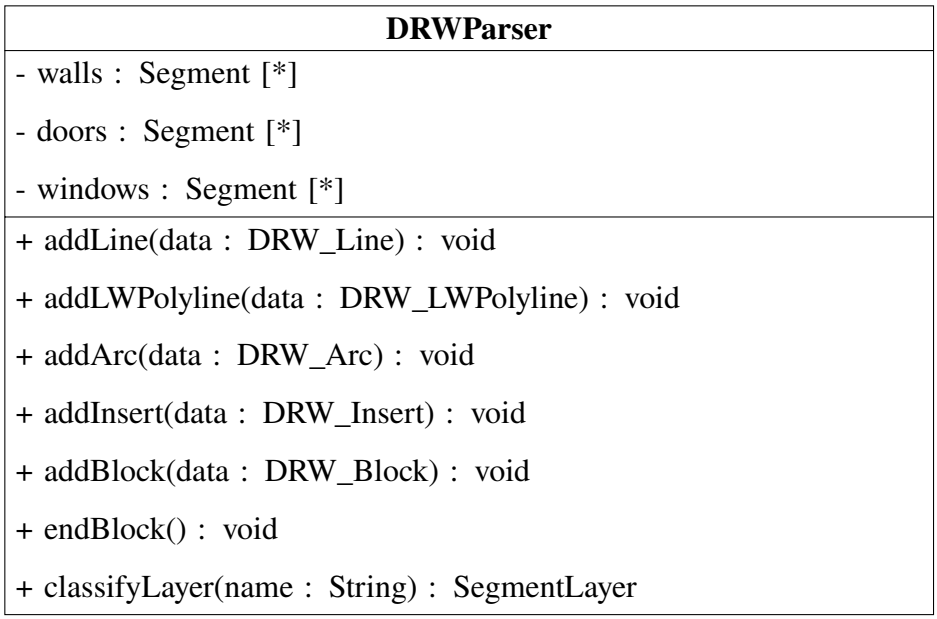


Figura 3.5: Diagramma UML della classe `DRWParser`.

Tra le callback messe a disposizione, quelle di maggiore interesse sono:

- `addLine()`: viene invocata per ogni segmento (`LINE`) presente nel disegno. I due punti estremi vengono memorizzati direttamente come segmento;
- `addLWPolyline()`: per le polilinee (`LWPolyline`), si iterano i vertici della polilinea per generare una sequenza di segmenti consecutivi;
- `addArc()`: per gli archi (`ARC`), si estraggono il raggio, il punto base, l'angolo iniziale e l'angolo finale; a partire da questi dati si calcolano i due punti estremi dell'arco, corrispondenti agli spigoli delle pareti adiacenti, e il segmento che li collega costituisce la rappresentazione della porta;
- `addInsert()`, `addBlock()`: gestiscono le istanze dei blocchi. Poiché le primitive all'interno di un blocco sono definite in coordinate locali, è necessario applicare una trasformazione (posizione, scala, rotazione) per riportarle in world space.

Per distinguere i segmenti in base alla loro semantica si utilizza il nome del layer ((Algoritmo 1)):

Algorithm 1 Classificazione del layer di un segmento

```
1:  function ClassifyLayer(name)
2:      if name = "A-DOOR" then
3:          return Door
4:      else if name = "A-WALL" then
5:          return Wall
6:      else if name = "A-WINDOW" then
7:          return Window
8:      else
9:          return None
10:     end if
11: end function
```

La stessa logica di corrispondenza per nome viene applicata al riconoscimento dei blocchi (`BLOCK-DOOR`, `BLOCK-WINDOW`) all'interno di `addBlock()`; proprio per questo motivo, rinominare correttamente layer e blocchi durante il preprocessing è un passaggio imprescindibile per la corretta riuscita del parsing.

Al termine di questa fase si ottengono tre vettori, contenenti rispettivamente le primitive (sotto forma di segmenti) relative a muri, porte e finestre.

Per verificare la correttezza del parsing si utilizza uno script Python che effettua il plot dei segmenti estratti (Figura 3.6).

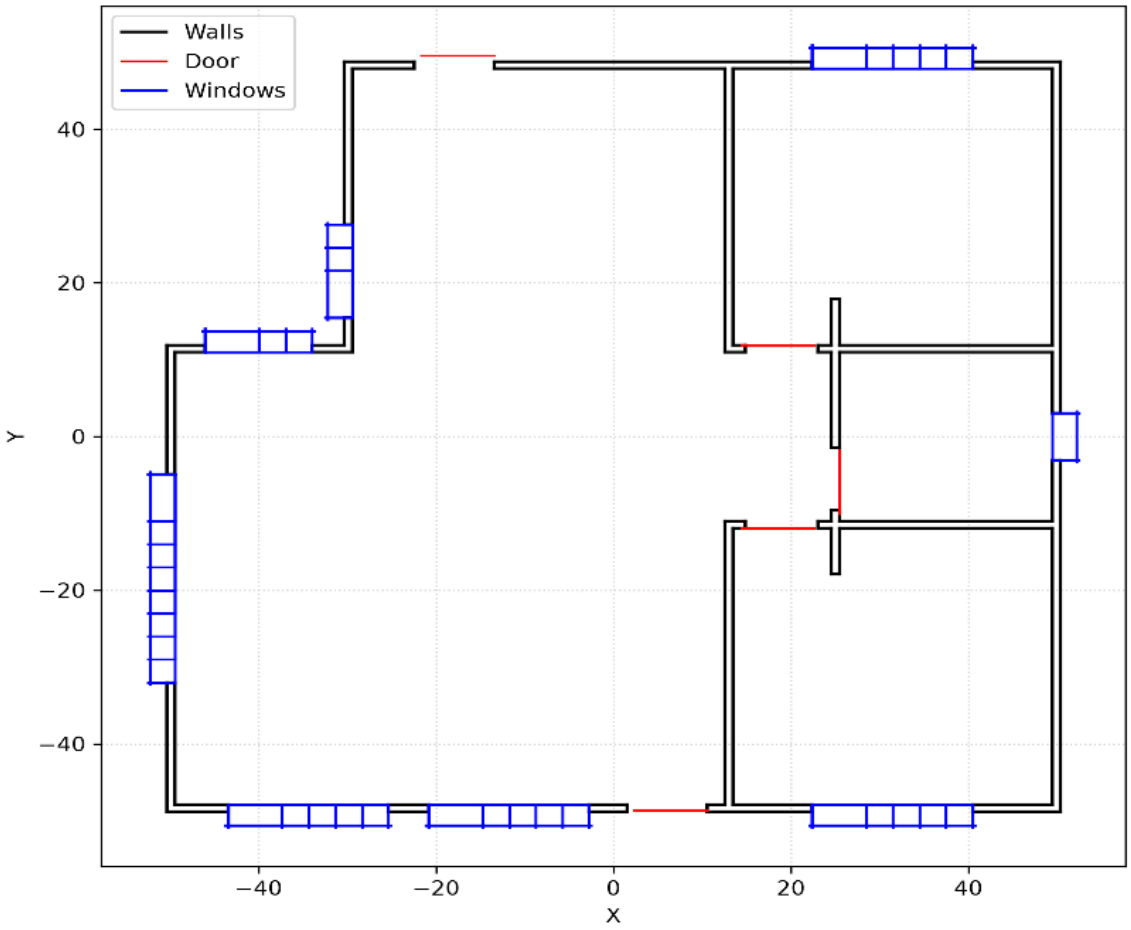


Figura 3.6: Risultato del parsing: le primitive geometriche estratte dal file DXF sono state collassate in segmenti e visualizzate tramite uno script Python per verificarne la correttezza.

3.4 Collasso dei vertici

Al termine della fase di parsing si dispone di una collezione di primitive (segmenti) indipendenti tra loro. Nei modelli CAD è abbastanza comune che in uno stesso punto siano presenti più vertici sovrapposti, appartenenti a segmenti diversi. È quindi necessario effettuare il collasso dei vertici, comunemente noto come *vertex snapping*. Questa operazione serve sia a ottimizzare la computazione, riducendo il numero di vertici da elaborare, sia a soddisfare il requisito, necessario per le fasi successive, di rappresentare la geometria come un grafo con vertici univoci.

È importante notare che in questa fase vengono considerati solamente i segmenti relativi ai muri: porte e finestre saranno gestite in maniera diversa più avanti (Sezione 3.5).

L'obiettivo è quindi prendere l'insieme dei segmenti e produrre una nuova rappresentazione in cui i vertici estremi che distano meno di ϵ l'uno dall'altro vengono collassati in un unico vertice rappresentativo. Per risolvere questo problema è necessaria una struttura dati spaziale efficiente; si è scelto di utilizzare un *hash spaziale*, discusso in dettaglio nella Sezione 2.1.

In questa fase è fondamentale tarare in modo adeguato il parametro ϵ: un valore troppo grande rischia di collassare erroneamente vertici distinti (ad esempio due angoli ravvicinati ma geometricamente reali), mentre un valore troppo piccolo lascia vertici duplicati che possono compromettere la costruzione del grafo nella fase successiva.

La classe che implementa questa struttura è mostrata nella Figura 3.7:

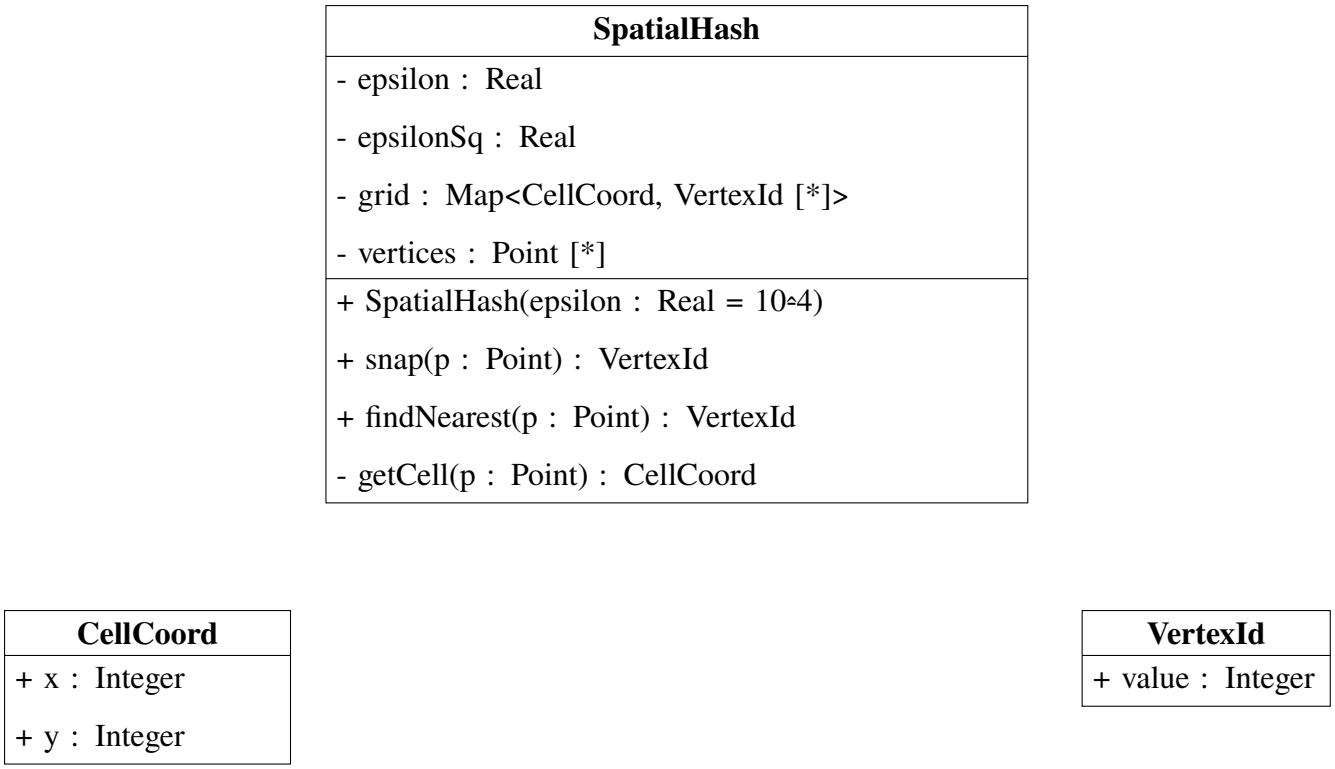


Figura 3.7: Diagramma UML della classe `SpatialHash`, che implementa la struttura dati spaziale usata per il collasso dei vertici.

Il parametro ϵ definisce il raggio di ricerca r e, di conseguenza, la dimensione della cella della griglia (Sezione 2.1.2). Il metodo principale è `snap()`: dato un punto p, cerca nella griglia 3×3 circostante un vertice già esistente che disti meno di ϵ; se lo trova, restituisce l'id (`VertexId`) del vertice esistente, sul quale il nuovo punto viene collassato senza essere inserito nella griglia; in caso contrario, il punto viene inserito come nuovo vertice e ne viene restituito l'id appena assegnato.

Il metodo `find_nearest()` restituisce invece l'id del vertice più vicino a p all'interno dell'intero hash, indipendentemente dalla distanza; questo metodo verrà utilizzato più avanti nella ricostruzione dei varchi (Sezione 3.5).

Utilizzando $\epsilon = 10^{-2}$, il numero di vertici sui segmenti dei muri del caso di studio si riduce da 177 a 88, ovvero una riduzione di circa il 50%. Riducendo ulteriormente il valore di ϵ, la riduzione ottenuta non varia in modo significativo, a conferma che il valore scelto è già sufficiente a collassare correttamente tutti i vertici duplicati presenti nel modello.

Un esempio di utilizzo della struttura (Algoritmo 2), nella costruzione dell'insieme di archi (`Edge`) a partire dai segmenti dei muri, è il seguente:

Algorithm 2 Costruzione degli archi a partire dai segmenti, con collasso dei vertici

Require: $segmentList$: lista dei segmenti estratti dal parsing

Require: $snapEps$: tolleranza ϵ per il collasso dei vertici

Ensure: $edges$: lista degli archi con vertici univoci

$hash \leftarrow$ SpatialHash($snapEps$)
$edges \leftarrow \emptyset$
for each $seg \in segmentList$ **do**
    $v_1 \leftarrow hash$.Snap($seg.start$)
    $v_2 \leftarrow hash$.Snap($seg.end$)
    if $v_1 \neq v_2$ **then**
        $edges$.Append($(v_1, v_2, seg.layer)$)
    end if
end for
return $edges$

Il controllo `v1 != v2` evita di introdurre archi degeneri, ovvero archi di lunghezza nulla generati da segmenti i cui due estremi vengono collassati sullo stesso vertice.

La struttura di un arco (`Edge`) è molto simile alla struttura `Segment` introdotta nella Sezione 3.3, con la differenza che, in questo caso, vengono memorizzati solo i riferimenti (id) ai vertici anziché le loro coordinate (Figura 3.8):

Edge
+ v1 : VertexId
+ v2 : VertexId
+ layer : SegmentLayer

Figura 3.8: Diagramma UML della struttura `Edge`, che rappresenta un lato del grafo con la relativa semantica.

Al termine di questa fase si dispone quindi di un vettore di `Edge` e di un vettore di vertici univoci.

3.5 Ricostruzione dei varchi

3.5.1 Ricostruzione porte

Il caso delle porte è il più semplice tra i due. Come si osserva dal plot ottenuto al termine del parsing (Figura 3.6), i segmenti delle porte collegano sempre due muri. Per ciascun segmento è necessario ricostruire un poligono semplice (una faccia chiusa), sfruttando la struttura spaziale e la lista degli archi (`Edge`) costruite nella fase precedente.

Si ricorda che il collasso dei vertici (Sezione 3.4) è stato effettuato sui soli segmenti dei muri: questo significa che sia i vertici sia gli archi presenti nell'hash spaziale appartengono esclusivamente ai muri.

L'idea è di iterare su ogni segmento delle porte, prelevarne le due estremità e cercare, nella struttura spaziale, i due vertici dei muri più vicini, che rappresentano i punti di giunzione. A tale scopo si utilizza il metodo `find_nearest()`, che restituisce proprio il riferimento al vertice più vicino. In questo modo si recuperano le due giunzioni e si crea un nuovo `Edge`, questa volta con tag `DOOR`.



Figura 3.9: Grafo dei muri e nuovo arco (tag `DOOR`) tra i vertici C e D, individuati come i vertici dei muri più vicini alle estremità del segmento della porta.

Per ottenere un poligono semplice è necessario inserire un secondo arco, parallelo al primo, tra due ulteriori vertici A e B.

Il criterio per individuare A e B sfrutta l'informazione di adiacenza già presente nel grafo dei muri: dati i due vertici di giunzione C e D e il versore (vettore normalizzato) $\vec{CD}$, si considerano i vertici adiacenti a C nel grafo — nell'esempio, A e F — e si verifica quale dei due, unito a C, individua un vettore ortogonale a $\vec{CD}$. Il test si effettua tramite il prodotto scalare tra versori: se il risultato è 0, i due vettori sono perpendicolari; altrimenti se è -1 sono invece opposti.

Applicando il test al vertice F si ottiene

$$\vec{CF} \cdot \vec{CD} = -1,$$

mentre per il vertice A

$$\vec{CA} \cdot \vec{CD} = 0 :$$

il vertice A è quindi il primo vertice candidato. Ripetendo lo stesso procedimento a partire da D, con versore $\vec{DC}$, sul vertice H si ottiene

$$\vec{DH} \cdot \vec{DC} = -1,$$

mentre per il vertice B

$$\vec{DB} \cdot \vec{DC} = 0 :$$

il vertice B è quindi il secondo vertice candidato. A questo punto si inserisce il secondo arco (tag `DOOR`), parallelo a $\vec{CD}$, corrispondente al segmento $\vec{AB}$.



Figura 3.10: Poligono semplice risultante dopo l'inserimento del secondo arco parallelo $\vec{AB}$, che chiude la faccia rappresentante la porta.

In questo modo non viene inserito alcun vertice nuovo, ma unicamente due nuovi archi (`Edge`).

3.5.2 Ricostruzione finestre

Nel modello di caso di studio una finestra è rappresentata da decine di segmenti e punti; in altri modelli, tuttavia, le finestre possono presentarsi in forme completamente diverse. Non essendo possibile trattare esplicitamente ogni possibile scenario di rappresentazione, è necessario un approccio più generale rispetto a quello utilizzato per le porte.

L'approccio adottato si basa sul clustering spaziale (Sezione 2.2), mediante l'algoritmo DBSCAN — per il quale si è utilizzata la libreria SimpleDBSCAN [6] — seguito dal calcolo del bounding box di ciascun cluster. Si assume che ogni finestra corrisponda a

un cluster indipendente. Per ciascun cluster viene calcolato il bounding box, da cui si estrae uno dei due lati lunghi: tale segmento congiunge le due giunzioni con i muri, ed è quindi possibile applicare lo stesso procedimento descritto in precedenza per ricostruire il secondo segmento parallelo e completare il poligono semplice. L'unica differenza rispetto al caso delle porte è che gli archi vengono taggati `WINDOW` anziché `DOOR`.

È cruciale, in questa fase, impostare correttamente il numero di campioni per ciascuna finestra e la distanza massima ϵ richiesti da DBSCAN. Nel caso di studio considerato si è impostato num_samples = 15 e $\epsilon = 2$ (Figura 3.11).

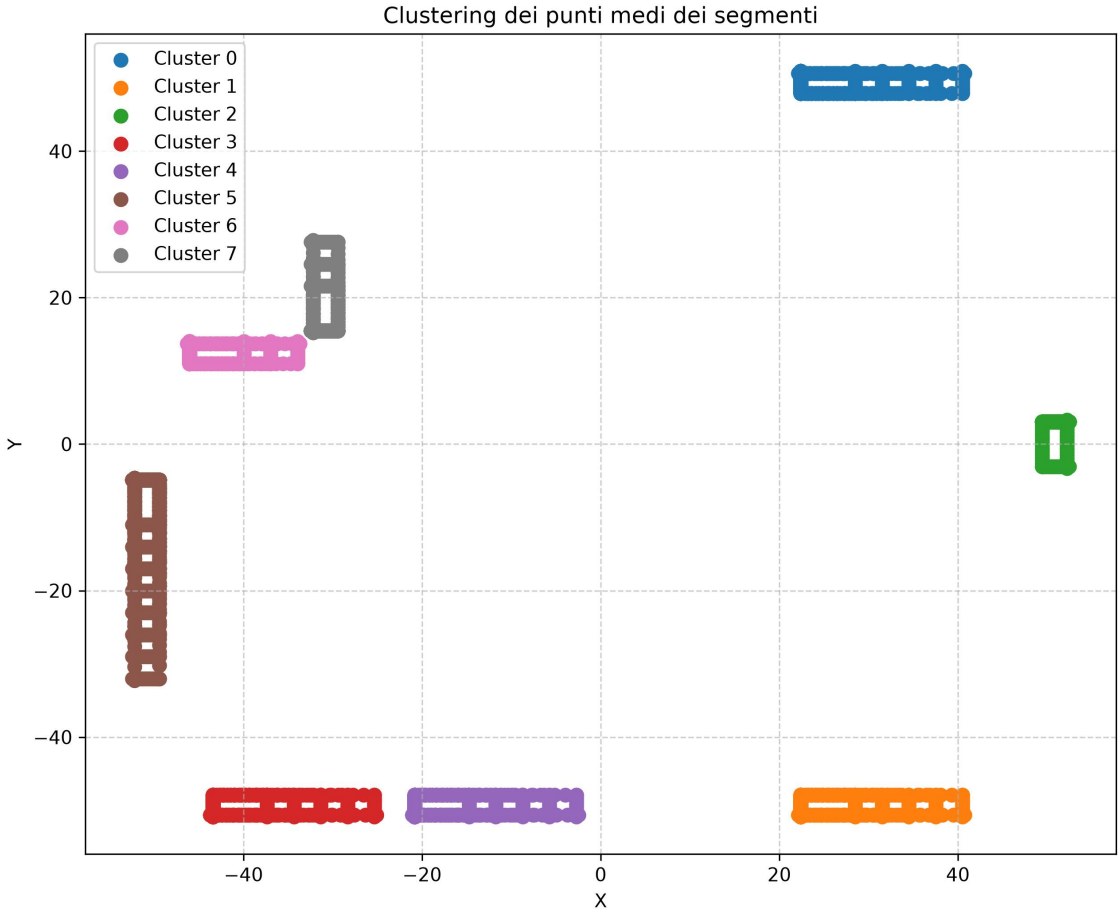


Figura 3.11: Cluster dei punti delle finestre individuati da DBSCAN con num_samples = 15 e $\epsilon = 2$.

A partire dai cluster individuati si calcolano i bounding box e si ricostruiscono i poligoni corrispondenti, ottenendo la rappresentazione finale (Figura 3.12).

Al termine di questa fase i vertici univoci presenti nell'hash spaziale restano invariati: viene aggiornato solamente il vettore degli `Edge`.

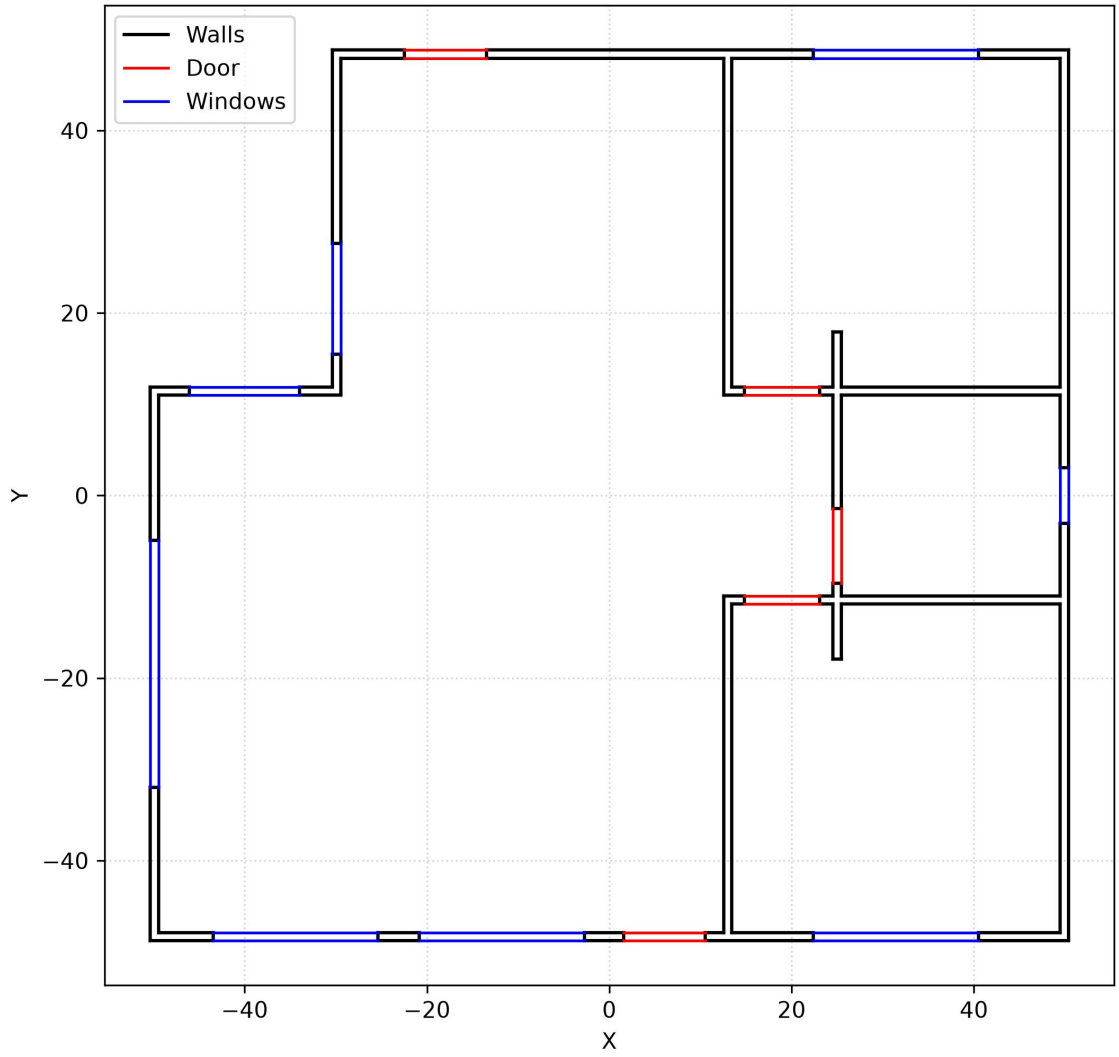


Figura 3.12: Rappresentazione finale del grafo dei muri con porte e finestre ricostruite e collegate alla struttura esistente.

3.6 Estrazione e classificazione delle facce

In questa fase l'obiettivo è rappresentare la geometria (vertici e archi) sotto forma di grafo planare embedded (PSLG) mediante la struttura Half-Edge, ed estrarre da essa le facce poligonali che costituiranno la base per la successiva triangolazione ed estrusione (Sezione 3.7).

Per implementare la struttura Half-Edge è stata utilizzata la libreria CGAL [12]. Come input si passano la lista dei vertici e degli archi (`Edge`); come output si ottiene una raccolta di poligoni 2D (facce) distinti ed espliciti.

A questo punto è possibile definire la struttura di una faccia (Figura 3.13), caratterizzata da: un vettore di vertici, i tipi di arco (layer) da cui è composta, e il tipo di faccia stesso:

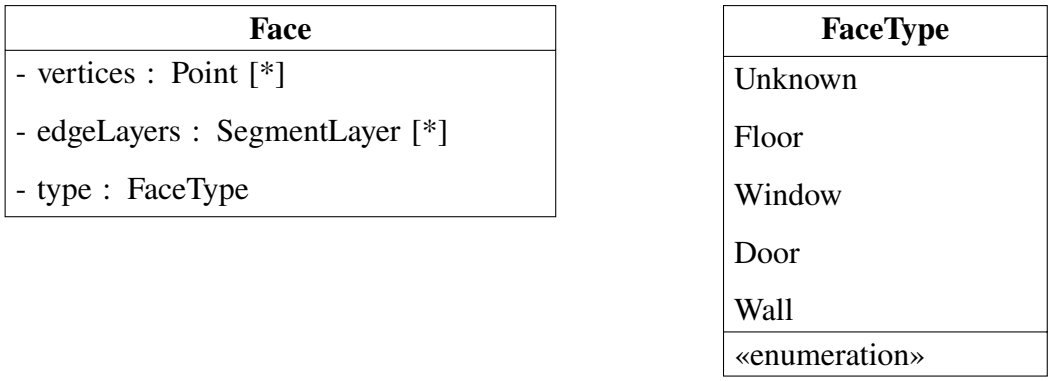


Figura 3.13: Diagramma UML della classe `Face`, che rappresenta una faccia del grafo con i relativi vertici, la semantica degli archi e il tipo di faccia.

Si itera quindi su ogni faccia estratta, effettuandone la classificazione in base al tipo dei suoi archi. Una faccia è classificata `DOOR` se e solo se ha esattamente quattro archi: 2 `DOOR` e 2 `WALL` (Figura 3.10). Analogamente, una faccia è classificata `WINDOW` se e solo se ha esattamente quattro archi: 2 `WINDOW` e 2 `WALL`. Una faccia è classificata `WALL` se tutti i suoi archi sono etichettati `WALL`. In tutti gli altri casi, la faccia resta non classificata.

La classificazione è fondamentale perché, come si vedrà nella Sezione 3.7, l'estrusione viene effettuata in modo differenziato a seconda del tipo di faccia.

Al termine di questa fase si effettua il plot delle facce estratte, per verificarne la correttezza (Figura 3.14).

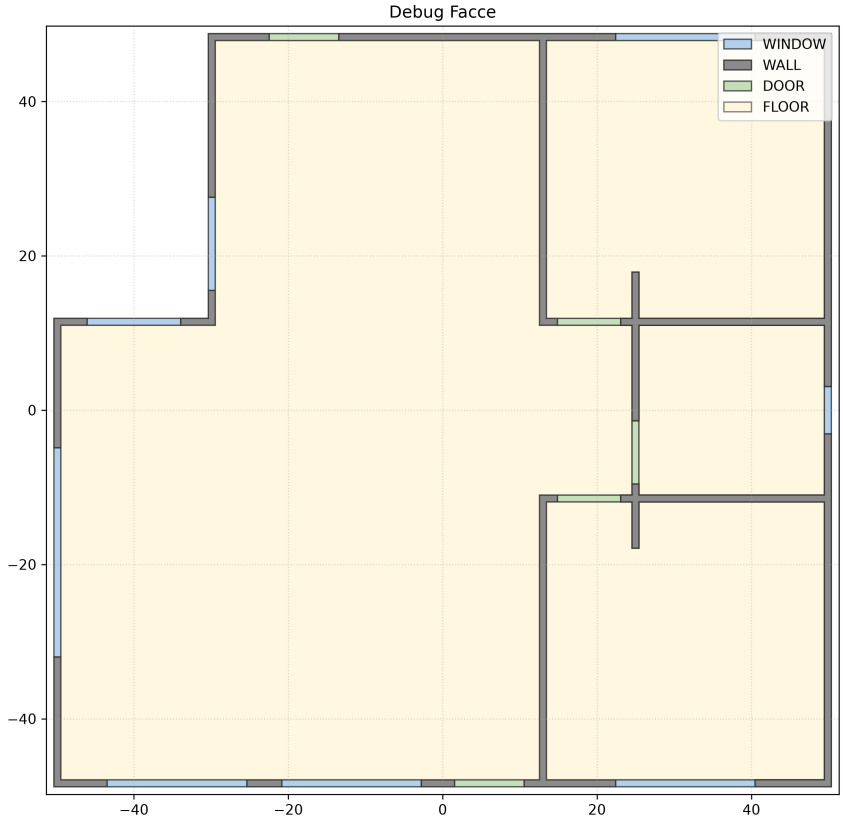


Figura 3.14: Facce poligonali estratte dalla struttura Half-Edge a partire dal PSLG dei muri, delle porte e delle finestre.

3.7 Ricostruzione mesh

In questa fase è necessario triangolare le facce estratte, calcolare gli attributi di ciascun vertice (posizione, normale e coordinate texture) ed estrudere le facce per ottenere infine una mesh tridimensionale.

Gli algoritmi di triangolazione sono stati discussi nella Sezione 2.4; in particolare è stata utilizzata la Constrained Delaunay Triangulation (CDT), tramite la libreria poly2tri [9]. Presa in input una faccia poligonale semplice, la libreria restituisce un vettore di triangoli. Per ciascun vertice di ciascun triangolo è quindi necessario calcolare posizione, normale e coordinate texture (Figura 3.15):

Vertex
+ position : Vec3
+ normal : Vec3
+ texCoord : Vec2

Figura 3.15: Diagramma UML della struttura `Vertex`, che rappresenta un vertice della mesh con posizione, normale e coordinate texture.

L'estrusione aggiunge profondità a una regione o faccia planare selezionata, lungo una direzione prestabilita. In pratica, occorre generare i vertici di una faccia ad altezze diverse e creare dei semplici quad verticali.

Un quad ha sempre quattro vertici: in basso a sinistra (*Bottom-Left*, BL), in basso a destra (*Bottom-Right*, BR), in alto a sinistra (*Top-Left*, TL) e in alto a destra (*Top-Right*, TR); di conseguenza, viene sempre suddiviso in esattamente due triangoli. Per ciascun vertice occorre calcolare il vettore normale, perpendicolare alla superficie del quad: l'ordine in cui si percorrono i vertici per formare i triangoli (*winding order*) ne determina il verso. Percorrendoli in senso antiorario (CCW) si ottiene una normale rivolta verso l'esterno, mentre percorrendoli in senso orario (CW) si ottiene una normale rivolta verso l'interno.

Le operazioni di triangolazione ed estrusione vengono applicate in modo diverso a seconda del tipo di faccia (pavimento, soffitto, muro, porta o finestra). Nel seguito, l'altezza del pavimento è indicata con 0 e quella del soffitto con H.

3.7.1 Pavimento e soffitto

Il procedimento per costruire il pavimento (e analogamente il soffitto) è semplice: si calcola il bounding box 2D dell'intera casa e si definisce un'unica faccia, grande quanto l'intera abitazione, posizionata all'altezza del pavimento (0), quindi triangolata. La normale è rivolta verso l'alto e le coordinate texture sono definite nell'intervallo $[0, 1]$.

Per il soffitto, la stessa faccia viene duplicata e traslata all'altezza H, con la normale rivolta verso il basso.

3.7.2 Porte

Una porta viene rappresentata come un muro pieno a partire dall'altezza dello stipite s fino al soffitto, ad altezza H; al di sotto dello stipite, l'apertura resta libera.

In questo caso, a differenza del muro pieno, è necessario applicare la triangolazione alla faccia orizzontale posta all'altezza dello stipite s, per evitare che vi si formi un buco visibile. Come per i muri, non è invece necessario triangolare le facce poste ad altezza pavimento né a quella posta ad altezza soffitto.

3.7.3 Finestre

La finestra è rappresentata come un'apertura al centro del muro: si distinguono quindi una porzione di muro inferiore, dal pavimento fino al davanzale (altezza d), e una porzione di muro superiore, dall'architrave (altezza a) fino al soffitto.

Sono pertanto necessarie due estrusioni distinte: una dall'altezza 0 all'altezza d, l'altra dall'altezza a all'altezza H; a differenza dei muri pieni, è inoltre necessario triangolare le facce orizzontali poste all'altezza del davanzale e dell'architrave. Come nei casi precedenti, non è invece necessario triangolare le facce poste ad altezza pavimento e ad altezza soffitto.

3.8 Esportazione

La mesh deve essere esportata in formato GLTF. Per questo compito è stata utilizzata la libreria tinygltf [11]. Vengono esportate di base le informazioni sui vertici: posizione, normale e coordinate texture.

Attualmente, porte e finestre sono rappresentate come semplici aperture nei muri: questo, tuttavia, non è sufficiente, poiché si vuole poter inserire, in corrispondenza di tali aperture, degli asset 3D di porte e finestre.

Una prima ipotesi era inserire gli asset direttamente nel processo di generazione della mesh, ma questo approccio risulta poco conveniente: se in un appartamento sono presenti dieci porte, la geometria della porta verrebbe duplicata altrettante volte, aumentando notevolmente la dimensione del file, tanto più se si considera che un modello dettagliato può contare centinaia di migliaia di triangoli. Inoltre, se si vuole cambiare asset o modificarne l'orientamento, sarebbe necessario rieseguire l'intera pipeline. Infine, occorrerebbe anche preservare i materiali originali dell'asset scaricato, complicando ulteriormente il processo.

La soluzione più flessibile e mantenibile consiste nell'adottare un approccio a placeholder, spostando il caricamento degli asset direttamente in Blender. L'idea è di esportare le informazioni sui varchi: quando una faccia viene riconosciuta come porta o finestra, se ne calcolano posizione, larghezza, altezza e spessore, oltre alla rotazione. Più precisamente, il centro rappresenta il centro dell'apertura, calcolato tenendo conto di stipite e davanzale: non si trova quindi all'altezza del soffitto, bensì esattamente a metà dell'apertura stessa.

Tutte queste informazioni vengono infine serializzate ed esportate sotto forma di file JSON. Ad esempio:

```
{
        "openings": [
        {
                "type": "DOOR",
                "center": [12.5, 0.0, 3.2],
                "width": 0.9,
                "height": 2.1,
                "thickness": 0.1,
                "rotation_z": 90.0
        },
        {
                "type": "WINDOW",
                "center": [8.3, 0.0, 1.5],
                "width": 1.2,
                "height": 1.4,
                "thickness": 0.1,
                "rotation_z": 0.0
        }]
}
```

4. Rendering

Per la fase di rendering viene utilizzato Blender, sia tramite uno script Python basato sulle API `bpy`, sia tramite la sua interfaccia grafica (GUI).

Il file GLTF esportato dalla pipeline (Sezione 3.8) contiene esclusivamente la geometria della mesh (posizioni, normali e coordinate texture) e non include alcuna informazione sui materiali. I materiali, così come gli asset di porte e finestre e l'environment map, vengono quindi definiti direttamente in Blender. In questo modo è possibile, ad esempio, cambiare le texture senza dover rieseguire ogni volta l'intera pipeline.

4.1 Generazione della scena

Inizialmente si era valutato di eseguire l'intero rendering all'interno dello script Python. Questo approccio si è tuttavia rivelato poco pratico: posizione e rotazione della camera, così come posizione e intensità delle luci, avrebbero dovuto essere codificate direttamente nel codice (*hard-coding*), rendendo la scelta dei parametri laboriosa e poco intuitiva. Si è quindi adottato un approccio ibrido: lo script Python automatizza le operazioni ripetitive e deterministiche, mentre la composizione finale della scena (camera e luci) è affidata alla GUI di Blender.

Lo script esegue, in ordine, le seguenti operazioni:

1. importazione del modello GLTF;
2. definizione dei materiali (albedo, normal, roughness, ecc.) per il pavimento e per i muri;
3. impostazione dell'environment map;
4. caricamento degli asset di porte e finestre e loro posizionamento in corrispondenza delle aperture, a partire dalle informazioni contenute nel file JSON dei varchi (Sezione 3.8);
5. creazione di alcune luci iniziali;
6. salvataggio della scena in un file `.blend`.

Il posizionamento degli asset si basa sul file `*_openings.json`, generato in fase di esportazione insieme al modello GLTF (Sezione 3.8). Per ciascun varco, il file contiene tutte le informazioni necessarie: il tipo (`DOOR` o `WINDOW`), il centro, la larghezza, l'altezza, lo spessore e la rotazione attorno all'asse verticale (`rotation_z`). Lo script utilizza il tipo per scegliere quale asset caricare, il centro e la rotazione per posizionarlo e orientarlo, e le dimensioni per scalarlo in modo che si adatti esattamente all'apertura.

4.2 Setup della scena e rendering

Il file `.blend` generato viene aperto in Blender tramite GUI, e comprende mesh, asset, materiali ed environment map. In questa modalità, l'impostazione della camera risulta molto più comoda: è possibile navigare liberamente nella scena, posizionare opportunamente le luci e modificarne le proprietà.

Il rendering fotorealistico viene eseguito con il motore Cycles, un path tracer fisicamente basato che simula il trasporto della luce nella scena, compresa l'illuminazione globale. Il rendering è stato effettuato a risoluzione 1280×720, con 128 campioni per pixel e denoiser attivo, il quale riduce il rumore residuo tipico dei metodi Monte Carlo e consente di utilizzare un numero contenuto di campioni.

Il rendering è stato eseguito esclusivamente su CPU (AMD Ryzen 5 2600X), poiché la GPU a disposizione (AMD Radeon RX 590) non risulta utilizzabile per il path tracing con Cycles. Anche il denoiser utilizzato, OpenImageDenoise, viene eseguito su CPU ed è impostato in qualità *High*. Il tempo di rendering varia in funzione della complessità della scena: una scena vuota con poche luci richiede circa 3 minuti, mentre l'aggiunta di asset e sorgenti luminose lo porta a circa 6-7 minuti.

4.3 Risultati

La Figura 4.1 mostra il rendering della scena base: la geometria generata dalla pipeline, con materiali, asset di porte e finestre ed environment map.

Come si osserva, la scena risulta piuttosto spoglia. Grazie alla modalità GUI è possibile aggiungere facilmente ulteriori asset per arredarla, come un divano, una TV, una lampada o un letto. Le figure seguenti mostrano alcuni esempi di scene arredate.



Figura 4.1: Rendering della scena base con Cycles (1280 × 720, 128 campioni per pixel, denoiser attivo).



Figura 4.2: Scena arredata: soggiorno con divano, TV, lampada



Figura 4.3: Scena arredata: camera da letto con una scrivania e un letto



Figura 4.4: Scena arredata: un bagno con un lavandino, uno specchio, un asciugamano

Tabella 4.1: Statistiche dell'esecuzione della pipeline.

Segmenti duplicati rimossi (finestre)	64
Segmenti totali (muro / porta / finestra)	88 / 4 / 113
Vertici 2D totali (tutti i segmenti)	410
Vertici sui muri prima dello snapping	176
Vertici sui muri dopo lo snapping	88
Vertici collassati	88
Vertici univoci nel grafo finale (2D)	88
Punti campionati per il clustering	3503
Cluster individuati (finestre)	8
Facce di muro	10
Facce di porta	4
Facce di finestra	8
Vertici mesh (3D)	804
Indici mesh	1140
Tempo di esecuzione della pipeline	5,7 s

5. Casi di studio e risultati

Nei capitoli precedenti la pipeline è stata testata su un primo modello utilizzato come caso di studio per descriverne le singole fasi.

Adesso vediamo come si comporta la pipeline su altri modelli e quali altri problemi possono esistere.

5.1 Secondo caso di studio

Il secondo modello considerato è mostrato in Figura 5.1.

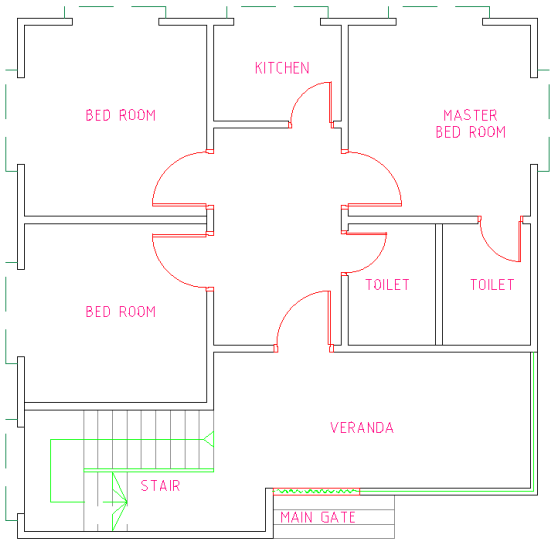


Figura 5.1: Modello CAD del secondo caso di studio prima del preprocessing.

Il preprocessing consiste nel rinominare i layer e i blocchi secondo la convenzione imposta e nel rimuovere quelli non utili. Il risultato è mostrato in Figura 5.2.

Rimane infine da eliminare il rumore (in verde) presente all'interno del rettangolo della porta d'ingresso.

La pipeline viene eseguita con i seguenti parametri:

- `unit scale` = 0.1
- `snap eps` = 0.01 (tolleranza ϵ per il collasso dei vertici, Sezione 3.4);
- `cluster num samples` = 30 e `cluster eps` = 0.5 (parametri di DBSCAN per il riconoscimento delle finestre, Sezione 3.5).

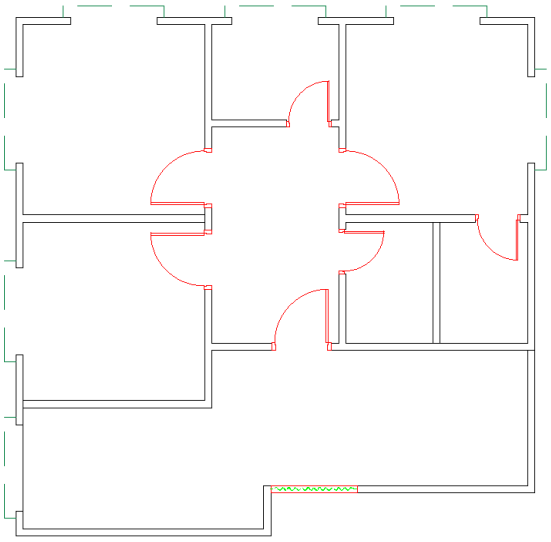


Figura 5.2: Modello CAD dopo la rinomina di layer e blocchi e la rimozione degli elementi non utili. In verde, il rumore presente all'interno del rettangolo della porta d'ingresso.

Come si osserva in Figura 5.3, il risultato non è ancora corretto: una faccia di muro non viene classificata correttamente e, in corrispondenza della porta d'ingresso, sono presenti segmenti che si intrecciano.

Il plot delle facce consente di individuare le due cause, entrambe risolvibili intervenendo sul modello CAD in LibreCAD.

Prima correzione. Nella porzione di muro relativa alla faccia non classificata sono stati rimossi due segmenti e ne sono stati aggiunti due nuovi, così da chiudere correttamente il contorno (Figura 5.4).

Seconda correzione. La rappresentazione della porta d'ingresso non è conforme alle assunzioni progettuali (Sezione 3.1): le due facce che delimitano il varco non risultano allineate al muro, ma sono rivolte una verso l'alto e l'altra verso sinistra. Dopo la correzione, la porta assume la forma mostrata in Figura 5.5.

Dopo le due correzioni la rappresentazione risulta completa e corretta (Figura 5.6); la Tabella 5.1 riporta le statistiche dell'esecuzione.

A partire da queste facce è possibile ricostruire il modello ed effettuare il rendering (Figura 5.7).

Il secondo caso di studio evidenzia che l'osservazione dei plot intermedi, in particolare quello delle facce, è fondamentale per individuare gli errori e capire se intervenire sul

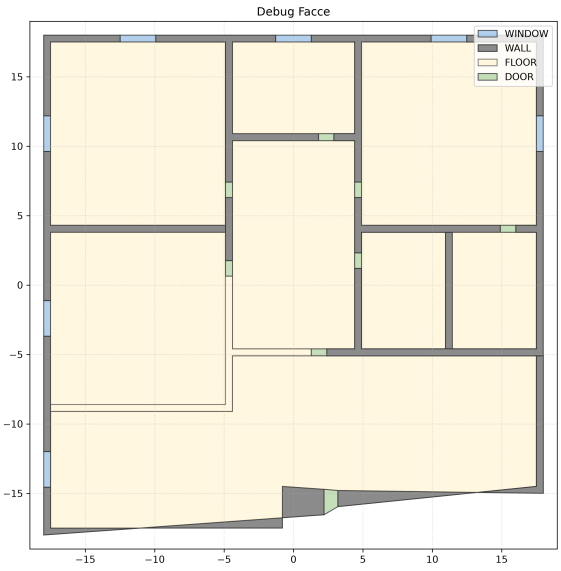


Figura 5.3: Facce estratte al primo tentativo: una faccia di muro non è classificata correttamente e sono presenti segmenti intrecciati in corrispondenza della porta d'ingresso.

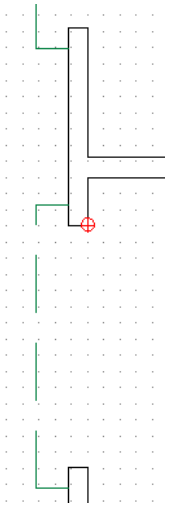


Figura 5.4: Prima correzione: sostituzione di due segmenti sulla porzione di muro non classificata.

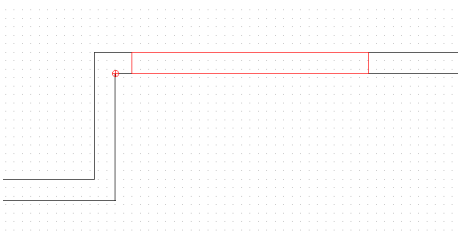


Figura 5.5: Seconda correzione: ridisegno della porta d'ingresso in modo conforme alle assunzioni progettuali.

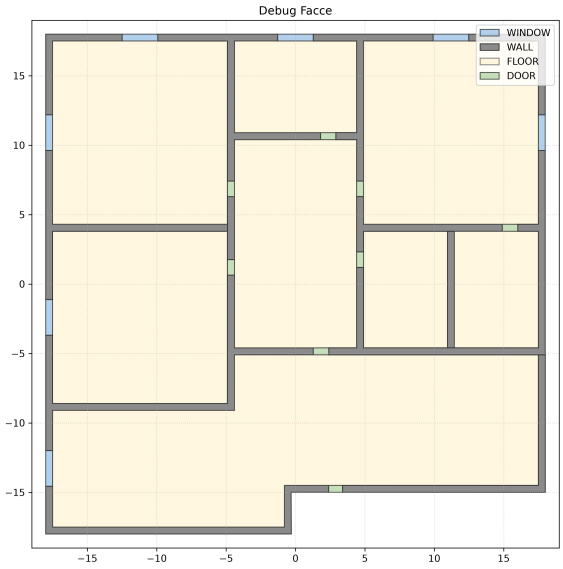


Figura 5.6: Facce estratte dopo le correzioni: la classificazione è corretta.



Figura 5.7: Rendering (1) del secondo caso di studio (1280×720, 128 campioni per pixel).



Figura 5.8: Rendering (2) del secondo caso di studio



Figura 5.9: Rendering (3) del secondo caso di studio

modello CAD o sui parametri. In questo caso è stato sufficiente un intervento minimo, la sostituzione di due segmenti e il ridisegno della porta d'ingresso, senza alcuna modifica al codice.

Ciò conferma però quanto discusso nella Sezione 3.1: la qualità del risultato dipende dalla conformità del modello di input alle assunzioni progettuali, e quando queste non sono soddisfatte la correzione resta un'operazione manuale.

Tabella 5.1: Statistiche dell'esecuzione della pipeline sul secondo caso di studio.

Segmenti duplicati rimossi (muri / porte)	2 / 1
Segmenti totali (muro / porta / finestra)	102 / 8 / 21
Vertici 2D totali (tutti i segmenti)	262
Vertici sui muri prima dello snapping	204
Vertici sui muri dopo lo snapping	105
Vertici collassati	99
Vertici univoci nel grafo finale (2D)	105
Punti campionati per il clustering	651
Cluster individuati (finestre)	7
Facce di muro	12
Facce di porta	8
Facce di finestra	7
Vertici mesh (3D)	948
Indici mesh	1350
Tempo di esecuzione della pipeline	5,9 s

5.2 Terzo caso di studio

Il terzo modello considerato è `pincad_4BHKDuplexHousePlan.dxf` (Figura 5.10).

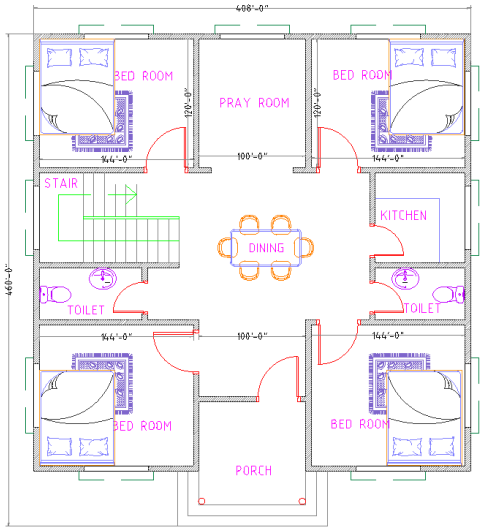


Figura 5.10: Modello CAD del terzo caso di studio.

In questo caso l'unica modifica necessaria consiste nella creazione dei varchi per le finestre: aprendo il modello in LibreCAD si osserva infatti come i muri risultino continui, senza alcuna apertura in corrispondenza delle finestre, in violazione dell'assunzione progettuale secondo cui porte e finestre devono essere rappresentate come aperture nei muri (Sezione 3.1). Il modello, una volta corretto, si presenta come mostrato in Figura 5.11.

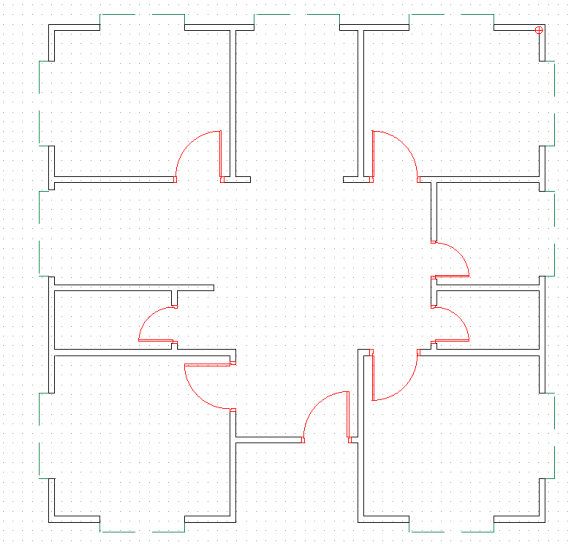


Figura 5.11: Modello CAD dopo la creazione manuale dei varchi per le finestre.

Con questa unica correzione, tutte le facce vengono classificate correttamente (Figura 5.12).

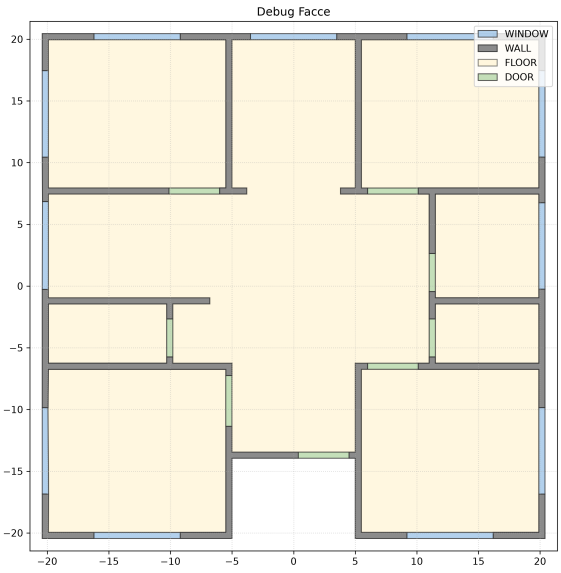


Figura 5.12: Facce estratte e classificate correttamente dopo la correzione.

Il rendering della scena, ottenuto con gli stessi parametri utilizzati nei casi precedenti (1280×720, 128 campioni per pixel), è mostrato in Figura 5.13.

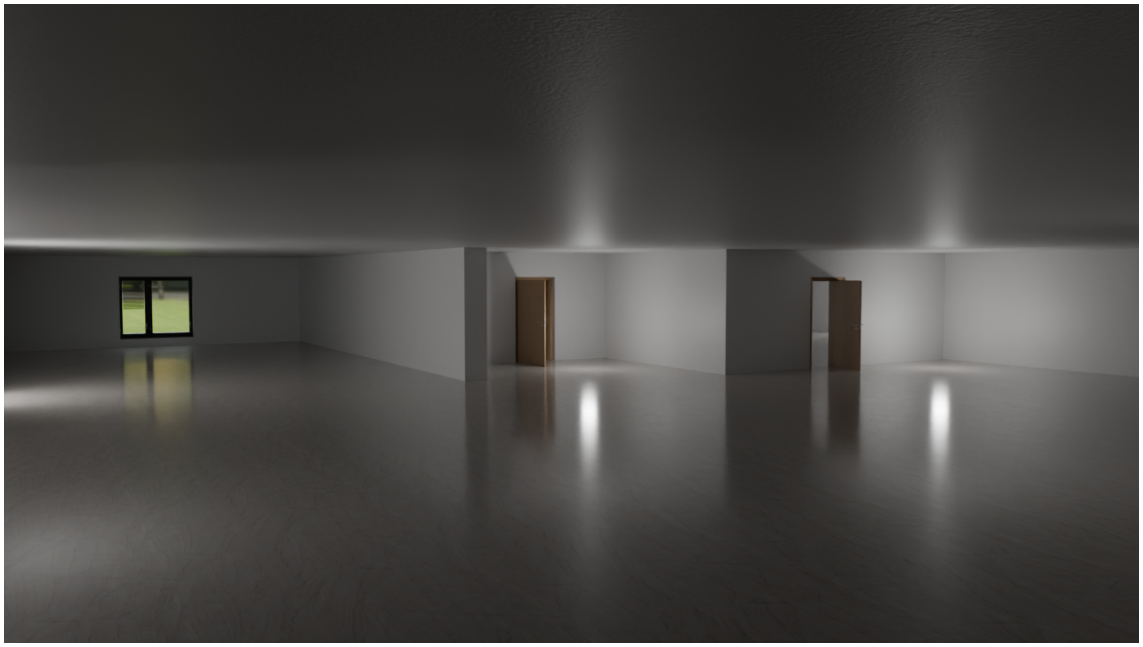


Figura 5.13: Rendering (1) del terzo caso di studio (1280×720, 128 campioni per pixel).

Rispetto ai casi precedenti, questo modello richiede un intervento manuale minimo e l'introduzione di elementi (i varchi) del tutto assenti nel disegno originale. Ciò conferma



Figura 5.14: Rendering (2) del terzo caso di studio

come il tipo di intervento richiesto dal preprocessing dipenda fortemente dalla sorgente e dalle convenzioni del singolo modello CAD.

Tabella 5.2: Statistiche dell'esecuzione della pipeline sul terzo caso di studio.

Segmenti totali (muro / porta / finestra)	132 / 8 / 33
Vertici 2D totali (tutti i segmenti)	346
Vertici sui muri prima dello snapping	264
Vertici sui muri dopo lo snapping	132
Vertici collassati	132
Vertici univoci nel grafo finale (2D)	132
Punti campionati per il clustering	1023
Cluster individuati (finestre)	11
Facce di muro	12
Facce di porta	8
Facce di finestra	11
Vertici mesh (3D)	1200
Indici mesh	1704
Tempo di esecuzione della pipeline	6,0 s

6. Conclusioni

6.1 Sintesi del lavoro

Il lavoro presentato in questa tesi ha affrontato il problema della generazione automatica di mesh 3D a partire da planimetrie architettoniche CAD, con l'obiettivo di ridurre l'intervento manuale tipicamente richiesto per ottenere una rappresentazione 3D di una scena a partire dal relativo disegno 2D.

È stata realizzata una pipeline in grado di: interpretare le entità geometriche eterogenee di un file DXF, riconducendole a muri, porte e finestre; ricostruire la topologia della planimetria; generare una mesh 3D triangolata, completa di normali e coordinate texture; ed esportare il risultato in formato GLTF, pronto per essere arricchito ed elaborato in Blender ai fini del rendering fotorealistico.

La validazione su diverse planimetrie reali ha mostrato come la pipeline sia in grado di produrre, in tempi dell'ordine di pochi secondi, una ricostruzione geometrica corretta a fronte di un intervento manuale contenuto e circoscritto alla fase di preprocessing, la cui entità dipende dalla conformità del modello di input alle assunzioni progettuali definite in Sezione 3.1.

6.2 Limiti

L'approccio presenta alcuni limiti, in parte intrinseci alla natura del problema affrontato e in parte legati a scelte progettuali specifiche.

- **Dipendenza dal preprocessing manuale.** La pipeline richiede che il modello di input rispetti dell assunzioni progettuali (Sezione 3.1); quando queste non sono soddisfatte, la correzione del modello CAD resta un'operazione manuale.
- **Sensibilità ai parametri.** La qualità del risultato dipende dalla corretta taratura di alcuni parametri, come l'unità di misura, la tolleranza ϵ per il collasso dei vertici (Sezione 3.4) e i parametri di DBSCAN per il riconoscimento delle finestre (Sezione 3.5); tali parametri non sono al momento determinati automaticamente, ma vanno impostati per ciascun modello.

- **Copertura geometrica limitata.** La pipeline è stata progettata e validata su planimetrie composte da muri rettilinei disposti su un unico piano; geometrie più complesse, come muri curvi o edifici multipiano, non sono attualmente supportate.
- **Composizione della scena non automatizzata.** Il posizionamento di camera e luci, così come l'eventuale arredamento della scena, sono attualmente operazioni manuali svolte tramite GUI di Blender (Capitolo 4), e non fanno parte della pipeline.

6.3 Sviluppi futuri

I limiti individuati suggeriscono diverse direzioni di sviluppo futuro. Una prima estensione riguarda l'automazione, anche solo parziale, della fase di preprocessing, ad esempio tramite euristiche in grado di individuare e correggere automaticamente contorni aperti o discontinuità nei muri.

Sul fronte del rendering, il posizionamento della camera e delle luci potrebbe essere automatizzato tramite euristiche basate sulla geometria della stanza (ad esempio posizionando le luci in funzione della posizione delle finestre), così come l'inserimento di arredi selezionati e disposti automaticamente in base al tipo di stanza.

Infine, un'estensione naturale del lavoro riguarda l'ampliamento della copertura geometrica della pipeline, per supportare planimetrie con muri curvi o edifici composti da più piani, oggi esclusi dalle assunzioni progettuali su cui il sistema si basa.

Bibliografia

[1] Autoware Foundation. *Spatial Hash*. https://autowarefoundation.github.io/autoware.universe/main/common/autoware_utils/design/spatial-hash-design/.

[2] Jaan Tollander de Balsch. *Searching for Fixed-Radius Near Neighbors with Cell Lists Algorithm in Julia Language*. https://jaantollander.com/post/searching-for-fixed-radius-near-neighbors-with-cell-lists-algorithm-in-julia-language/. 2020.

[3] codelibs, Nicu Tofan e Miguel E. Hernández Cuervo. *libdxfrw: C++ library to read and write DXF/DWG files*. Fork del progetto originale libdxfrw su SourceForge. 2026. URL: https://github.com/codelibs/libdxfrw.

[4] David Eberly. *Triangulation by Ear Clipping*. Geometric Tools, Redmond WA 98052. 2002. URL: https://www.geometrictools.com/Documentation/TriangulationByEarClipping.pdf.

[5] Martin Ester et al. «A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise». In: *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD-96)*. AAAI Press, 1996, pp. 226–231.

[6] Z. J. Jiang. *SimpleDBSCAN: A one header only DBSCAN implementation with N-dimensions support*. 2024. URL: https://github.com/CallmeNezha/SimpleDBSCAN.

[7] Gary W. Lucas. *An Introduction to the Delaunay Triangulation*. TinfourDocs, Tinfour Project. 2016. URL: https://gwlucastrig.github.io/TinfourDocs/DelaunayIntro/.

[8] Steve Marschner e Peter Shirley. *Fundamentals of Computer Graphics*. 5ª ed. CRC Press, 2021.

[9] Poly2Tri Contributors. *poly2tri: 2D constrained Delaunay triangulation library*. 2024. URL: https://github.com/jhasse/poly2tri.

[10] scikit-learn developers. *sklearn.cluster.DBSCAN*. https://scikit-learn.org/stable/modules/generated/sklearn.cluster.DBSCAN.html.

[11] syoyo. *tinygltf: Header only C++11 tiny glTF 2.0 library*. Libreria header-only per il caricamento e il salvataggio di file glTF 2.0. 2026. URL: https://github.com/syoyo/tinygltf.

[12] The CGAL Project. *CGAL: The Computational Geometry Algorithms Library*. 2026. URL: https://github.com/CGAL/cgal.

[13] Okan Yenigün. *DBSCAN Clustering Algorithm Demystified*. https://builtin.com/articles/dbscan. 2024.